简易 Zig

JW Lee

简易 **Zig**

Zig 编程语言零基础入门指南

作者：*JW Lee*

前言

欢迎阅读《简易 Zig》！本书专为希望从零开始学习 Zig 编程语言的初学者设计。无论你来自 Python、JavaScript、C 语言背景，还是这是你的第一门编程语言，本书都会一步步指导你。

什么是 **Zig**？

Zig 是一种现代系统编程语言，由 Andrew Kelley 于 2016 年创建。它被设计为"更好的 C"——在提供对内存和硬件的底层控制的同时，避免了许多让 C 和 C++ 变得困难和容易出错的陷阱。

```
┌──────────────────────────────────────────────────────────┐
│                    编程语言光谱                           │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  高级语言                                    底层语言    │
│  (更简单)                                    (更快速)    │
│                                                          │
│  Python ─── JavaScript ─── Go ─── Zig ─── C ─── ASM     │
│     │           │           │       │       │       │    │
│  脚本语言    Web应用      服务     系统    操作系统  硬件  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

为什么选择 **Zig**？

1. 简洁性

与 C++ 不同，Zig 没有隐藏的控制流、没有隐藏的内存分配、没有运算符重载。所见即所得。

2. 无垃圾回收的安全性

Zig 在编译时捕获许多错误。默认情况下没有空指针，数组有边界检查，并且有显式的错误处理。

3. 性能

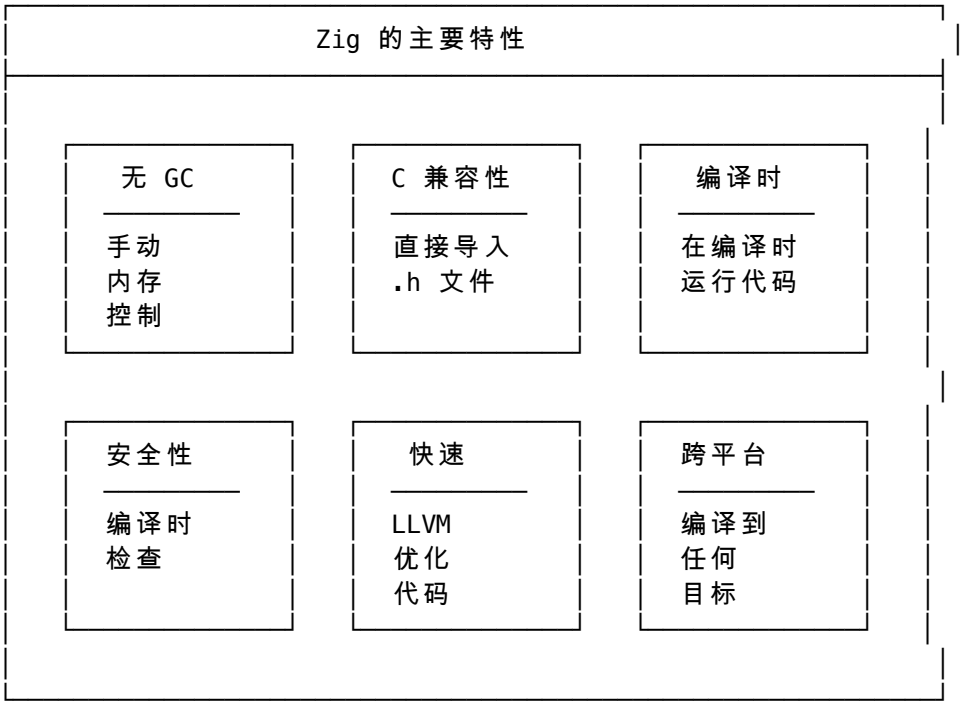
Zig 编译成高度优化的机器码，可与 C 和 C++ 媲美。Uber 等公司在生产环境中使用它。

4. C 语言互操作性

Zig 可以直接导入和使用 C 头文件。无需绑定。这意味着你可以访问数十年的 C 库。

5. 编译时执行 **(Comptime)**

Zig 可以在编译时运行代码，无需宏或模板即可实现强大的元编程。



安装 Zig

安装方法

macOS (Homebrew):

```
brew install zig
```

Linux:

```
# 从 ziglang.org 下载并解压
wget https://ziglang.org/download/zig-linux-x86_64.tar.xz
tar -xf zig-linux-x86_64.tar.xz
export PATH=$PATH:./zig-linux-x86_64
```

Windows: 从 https://ziglang.org/download/ 下载安装程序

验证安装

```
zig version
```

你的第一条命令

```
# 创建一个名为 hello.zig 的文件，然后运行：
zig run hello.zig
```

如何使用本书

每章介绍一个概念，包含：1. 解释 - 概念是什么以及为什么重要 2. 图表 - 工作原理的可视化表示 3. 代码示例 - 你可以自己运行的工作代码 4. 输出 - 运行代码时应该看到的内容 5. 要点 - 重要内容的总结

让我们开始你的 Zig 之旅！

第一章：Hello World

每个编程之旅都从 Hello World 开始。这个简单的程序教你 Zig 程序的基本结构。

理解程序结构

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 程序结构                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ┌──────────────────────────────┐                      │
│    │ 1. 导入                      │                      │
│    │    const std = @import("std");│                     │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 2. 常量和全局变量            │                      │
│    │    const MAX = 100;          │                      │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 3. 函数                      │                      │
│    │    fn helper() { ... }       │                      │
│    └──────────────────────────────┘                      │
│                    │                                     │
│                    ▼                                     │
│    ┌──────────────────────────────┐                      │
│    │ 4. 主函数（入口点）          │                      │
│    │    pub fn main() void { ... }│                      │
│    └──────────────────────────────┘                      │
│                                                          │
```

└──┘

你的第一个程序

```zig
// ============================================================
// 第一章：Hello World - 你的第一个 Zig 程序
// ============================================================

// 第一步：导入标准库
// @import 函数引入外部代码
// "std" 是 Zig 的标准库，包含有用的函数
const std = @import("std");

// 第二步：定义主函数
// - "pub" 表示公开（可以从外部调用）
// - "fn" 声明一个函数
// - "main" 是入口点 - 程序从这里开始
// - "void" 表示这个函数不返回任何东西
pub fn main() void {
    // 第三步：打印到控制台
    // std.debug.print() 将文本输出到终端
    // \n 创建一个新行
    // .{} 是用于格式参数的空元组
    std.debug.print("Hello, World!\n", .{});
}
```

输出：

```
Hello, World!
```

分解每个部分

导入语句

```zig
const std = @import("std");
```

这行代码做了三件事：1. `@import("std")` - 加载 Zig 的标准库 2. `const std` - 创建一个名为 `std` 的常量 3. `=` - 将库赋值给常量

可以这样理解：

```
┌──────────────────────────────────────────────────────────────────┐
│                        @import("std")                            │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   标准库内容：                                                    │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │   debug      │  │    mem       │  │    fmt       │          │
│   │  .print()    │  │   .copy()    │  │  .format()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐          │
│   │    heap      │  │    fs        │  │    os        │          │
│   │ .allocator() │  │ .openFile()  │  │  .system()   │          │
│   └──────────────┘  └──────────────┘  └──────────────┘          │
│                                                                  │
```

```
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

主函数

```
pub fn main() void {
    // 代码在这里
}
```

- pub - 使函数对运行时可见
- fn - 声明函数的关键字
- main - 被识别为程序入口点的特殊名称
- () - 空括号表示没有参数
- void - 返回类型（不返回任何东西）
- { } - 花括号包含函数体

打印语句

```
std.debug.print("Hello, World!\n", .{});
```

让我们分解一下：- std - 我们导入的标准库 - .debug - std 中用于调试工具的命名空间 - .print() - 输出格式化文本的函数 - "Hello, World!\n" - 格式字符串（要打印的文本）- \n - 换行转义序列 - .{} - 用于参数的空匿名结构体字面量（元组）

使用变量打印

现在让我们通过使用变量使程序更加动态：

```
const std = @import("std");

pub fn main() void {
    // 声明一个字符串常量
    // "Zig" 自动被类型化为指向字节的指针
    const name = "Zig";

    // 声明一个整数常量
    // : u32 明确指定类型为无符号 32 位
    const year: u32 = 2016;

    // 使用格式说明符打印：
    // {s} - 格式化为字符串
    // {d} - 格式化为十进制数
    std.debug.print("Welcome to {s}!\n", .{name});
    std.debug.print("{s} was created in {d}.\n", .{ name, year });

    // 元组中的多个值
    const version = "0.11";
    const is_stable = true;
    std.debug.print("Version: {s}, Stable: {}\n", .{ version, is_stable });
}
```

输出：

```
Welcome to Zig!
```

```
Zig was created in 2016.
Version: 0.11, Stable: true
```

格式说明符参考

格式说明符	
说明符	描述
{s}	字符串
{d}	十进制数（整数）
{x}	十六进制（小写）
{X}	十六进制（大写）
{b}	二进制
{c}	字符
{}	默认格式（自动检测）
{any}	打印任何类型（用于调试）
{?}	可选值
{e}	科学计数法
修饰符	
{d:5}	最小宽度为 5
{d:0>5}	右对齐，用零填充，宽度 5
{d:<5}	左对齐，宽度 5
{d:.2}	2 位小数（用于浮点数）

高级打印示例

```zig
const std = @import("std");

pub fn main() void {
    // 数字格式化
    const num: i32 = 42;
    const float: f64 = 3.14159265359;
    const byte: u8 = 255;

    std.debug.print("Decimal:     {d}\n", .{num});
    std.debug.print("Hexadecimal: 0x{x}\n", .{byte});      // 0xff
    std.debug.print("Binary:      0b{b}\n", .{byte});      // 0b11111111
    std.debug.print("Float:       {d:.2}\n", .{float});    // 3.14
    std.debug.print("Scientific:  {e}\n", .{float});       // 3.14159e+00

    // 填充和对齐
    std.debug.print("Right aligned: |{d:>10}|\n", .{num}); // |        42|
    std.debug.print("Left aligned:  |{d:<10}|\n", .{num}); // |42        |
    std.debug.print("Zero padded:   |{d:0>10}|\n", .{num});// |0000000042|
}
```

输出：

```
Decimal:     42
Hexadecimal: 0xff
Binary:      0b11111111
Float:       3.14
Scientific:  3.14159265359e0
Right aligned: |        42|
Left aligned:  |42        |
Zero padded:   |0000000042|
```

要点

1. 每个 **Zig** 程序都需要一个 **`main`** 函数 - 这是执行开始的地方
2. 先导入再使用 - 使用 `@import` 引入库
3. 格式字符串是类型检查的 - Zig 在编译时验证你的格式说明符与参数匹配
4. 需要分号 - 与 JavaScript 不同，你不能省略它们
5. 注释使用 `//` - 一行中 `//` 之后的所有内容都会被忽略

第二章：变量

变量是存储值的容器。Zig 对可以改变的值和不能改变的值有明确的区分。

两种类型的变量

```
┌──────────────────────────────────────────────────────────────┐
│                    Zig 中的变量类型                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────┐      ┌──────────────────┐            │
│   │      const       │      │       var        │            │
│   │    ( 不可变 )    │      │    ( 可变 )      │            │
│   ├──────────────────┤      ├──────────────────┤            │
│   │                  │      │                  │            │
│   │ const x = 5;     │      │ var x = 5;       │            │
│   │ x = 10; // 错误！ │      │ x = 10; // 可以！ │            │
│   │                  │      │                  │            │
│   │ • 不能改变       │      │ • 可以改变       │            │
│   │ • 首选           │      │ • 需要时使用     │            │
│   │ • 线程安全       │      │ • 要小心         │            │
│   │ • 可优化         │      │                  │            │
│   └──────────────────┘      └──────────────────┘            │
│                                                              │
│   规则：除非需要修改，否则始终使用 const！                    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

常量 (const)

常量是初始化后永远不会改变的值。它们是 Zig 中的默认选择。

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 常量 - 永远不会改变的值
    // ========================================

    // 类型推断（Zig 自己推断出来）
    const greeting = "Hello";           // []const u8（字符串）
    const answer = 42;                   // comptime_int
    const pi = 3.14159;                  // comptime_float

    // 类型显式指定（你指定它）
    const age: u32 = 25;                 // 无符号 32 位整数
    const temperature: f64 = 98.6;       // 64 位浮点数
    const is_valid: bool = true;         // 布尔值

    std.debug.print("greeting: {s}\n", .{greeting});
    std.debug.print("answer: {d}\n", .{answer});
    std.debug.print("pi: {d:.5}\n", .{pi});
    std.debug.print("age: {d}\n", .{age});
    std.debug.print("temperature: {d}\n", .{temperature});
    std.debug.print("is_valid: {}\n", .{is_valid});

    // 这会导致编译错误：
    // age = 26;  // 错误：不能赋值给常量
}
```

输出：

```
greeting: Hello
answer: 42
pi: 3.14159
age: 25
temperature: 98.6
is_valid: true
```

变量 (var)

变量可以在初始化后更改。只在必要时使用它们。

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 变量 - 可以改变的值
    // ========================================

    var counter: i32 = 0;
    std.debug.print("Initial counter: {d}\n", .{counter});

    // 修改变量
    counter = counter + 1;  // 现在是 1
    std.debug.print("After +1: {d}\n", .{counter});
```

```zig
    counter += 10;          // counter = counter + 10 的简写
    std.debug.print("After +=10: {d}\n", .{counter});

    counter *= 2;           // 翻倍
    std.debug.print("After *=2: {d}\n", .{counter});

    // 用 var 声明变量时必须有类型
    var name: []const u8 = "Alice";
    std.debug.print("Name: {s}\n", .{name});

    name = "Bob";           // 改变值
    std.debug.print("New name: {s}\n", .{name});
}
```

输出：

```
Initial counter: 0
After +1: 1
After +=10: 11
After *=2: 22
Name: Alice
New name: Bob
```

变量的内存布局

```
┌──────────────────────────────────────────────────────────────────────┐
│                              内存布局                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   当你写：var x: i32 = 42;                                            │
│                                                                      │
│   栈内存：                                                            │
│   ┌──────────────────────────────────────┐                           │
│   │  地址          │  值       │  变量      │                         │
│   ├──────────────┼──────────┼──────────┤                           │
│   │  0x1000       │   42      │     x      │ ← 4 字节 (i32)          │
│   │  0x1004       │   ...     │  (下一个)  │                         │
│   └──────────────┴──────────┴──────────┘                           │
│                                                                      │
│   不同类型使用不同数量的内存：                                          │
│   ┌──────────────────────────────────────────────────┐               │
│   │ 类型      │ 大小      │ 内存可视化                              │
│   ├──────────┼──────────┼──────────────────────────────────────────│
│   │ u8        │ 1 字节    │ [  ]                                    │
│   │ u16       │ 2 字节    │ [    ]                                  │
│   │ u32       │ 4 字节    │ [        ]                              │
│   │ u64       │ 8 字节    │ [                ]                      │
│   │ bool      │ 1 字节    │ [  ]                                    │
│   └──────────────────────────────────────────────────┘               │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

未定义值

有时你想声明一个变量但稍后再赋值。使用 undefined：

```
const std = @import("std");

pub fn main() void {
    // ========================================
    // UNDEFINED - 现在声明，稍后赋值
    // ========================================

    // 声明但不初始化
    // 警告：在赋值前使用未定义的值是未定义行为！
    var result: i32 = undefined;

    // ... 一些确定值的代码 ...

    // 现在赋值
    result = calculateSomething();

    std.debug.print("Result: {d}\n", .{result});
}

fn calculateSomething() i32 {
    return 42;
}
```

输出：

```
Result: 42
```

```
┌──────────────────────────────────────────────────────────┐
│                      UNDEFINED 警告                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   ⚠  危险：读取未定义的值是未定义行为 (UB)。              │
│       程序可能：                                         │
│                                                          │
│       • 崩溃                                             │
│       • 返回垃圾值                                       │
│       • 行为不可预测                                     │
│       • 在调试模式下工作但在发布模式下失败                │
│                                                          │
│   安全模式：                                             │
│   ┌──────────────────────────────────────┐               │
│   │ var x: i32 = undefined;              │               │
│   │ x = computeValue();  // 必须先赋值！  │               │
│   │ use(x);              // 现在可以安全使用 │           │
│   └──────────────────────────────────────┘               │
│                                                          │
│   不安全模式（不要这样做）：                              │
│   ┌──────────────────────────────────────┐               │
│   │ var x: i32 = undefined;              │               │
│   │ use(x);  // UB！x 有垃圾值            │               │
```

```
│    └──────────────────────────────────────────────┘              │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

块表达式

Zig 允许你使用块来计算复杂的初始值：

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 块表达式 - 复杂初始化
    // ========================================

    // 使用块来计算一个值
    const result = blk: {
        var temp: i32 = 10;
        temp *= 2;           // 20
        temp += 5;           // 25
        temp = temp * temp;  // 625
        break :blk temp;     // 返回值
    };

    std.debug.print("Result: {d}\n", .{result});

    // 对条件初始化很有用
    const value: i32 = 42;
    const description = blk: {
        if (value < 0) {
            break :blk "negative";
        } else if (value == 0) {
            break :blk "zero";
        } else {
            break :blk "positive";
        }
    };

    std.debug.print("{d} is {s}\n", .{ value, description });
}
```

输出：

```
Result: 625
42 is positive
```

编译时常量

以 comptime 为前缀或在顶层定义的常量在编译时求值：

```zig
const std = @import("std");

// ========================================
// 顶层常量（编译时）
// ========================================
```

```zig
// 这些在程序运行之前计算
const SECONDS_PER_MINUTE = 60;
const MINUTES_PER_HOUR = 60;
const HOURS_PER_DAY = 24;

// 从其他常量计算（仍然是编译时）
const SECONDS_PER_HOUR = SECONDS_PER_MINUTE * MINUTES_PER_HOUR;
const SECONDS_PER_DAY = SECONDS_PER_HOUR * HOURS_PER_DAY;

// 在编译时执行的函数
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// 预计算的阶乘
const FACT_5 = factorial(5);   // 在编译时计算！
const FACT_10 = factorial(10); // 也是编译时！

pub fn main() void {
    std.debug.print("Seconds per hour: {d}\n", .{SECONDS_PER_HOUR});
    std.debug.print("Seconds per day: {d}\n", .{SECONDS_PER_DAY});
    std.debug.print("5! = {d}\n", .{FACT_5});
    std.debug.print("10! = {d}\n", .{FACT_10});
}
```

输出：

```
Seconds per hour: 3600
Seconds per day: 86400
5! = 120
10! = 3628800
```

```
┌──────────────────────────────────────────────────────────────┐
│                 编译时 vs 运行时                              │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   源代码                                                     │
│       │                                                      │
│       ▼                                                      │
│   ┌──────────────────────────────────────────┐               │
│   │              编译时                       │               │
│   │   • 顶层的 const                          │               │
│   │   • comptime 块                           │               │
│   │   • 类型计算                              │               │
│   │   • 泛型实例化                            │               │
│   │                                           │               │
│   │   例子：const X = factorial(10);          │               │
│   │         现在计算！不是在运行时。          │               │
│   └──────────────────────────────────────────┘               │
│       │                                                      │
│       ▼                                                      │
│   可执行文件（X 已经是 3628800）                              │
```

```
│        │                                                  │
│        ▼                                                  │
│   ┌──────────────────────────────────────────────┐        │
│   │              运行时                            │        │
│   │   • var 变量                                   │        │
│   │   • 用户输入                                   │        │
│   │   • 带运行时值的函数调用                        │        │
│   │                                                │        │
│   │   例子：var x = getUserInput();                │        │
│   │         现在计算，在执行期间。                  │        │
│   └──────────────────────────────────────────────┘        │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

命名约定

```
┌──────────────────────────────────────────────────────────┐
│                      命名约定                             │
├──────────────────────────────────────────────────────────┤
│                                                           │
│   类型              │  约定            │  示例            │
│   ──────────────────────────────────────────────────────  │
│   局部变量          │  snake_case      │  my_variable     │
│   常量              │  snake_case 或   │  max_size        │
│                     │  SCREAMING_CASE  │  MAX_SIZE        │
│   函数              │  camelCase       │  calculateSum    │
│   类型/结构体       │  PascalCase      │  MyStruct        │
│   编译时常量        │  SCREAMING_CASE  │  BUFFER_SIZE     │
│                                                           │
│   示例：                                                  │
│   ┌──────────────────────────────────────────────┐        │
│   │  const BUFFER_SIZE = 1024;  // 编译时常量      │        │
│   │  var user_count: u32 = 0;   // 可变变量        │        │
│   │  const User = struct {};    // 类型定义        │        │
│   │  fn getUserName() {}        // 函数            │        │
│   └──────────────────────────────────────────────┘        │
│                                                           │
└──────────────────────────────────────────────────────────┘
```

要点

1. 优先使用 **const** 而不是 **var** - 不可变性可以防止错误
2. 类型可以推断或显式指定 - Zig 在可能时推断类型
3. **undefined** 是危险的 - 只在你会在读取之前赋值时使用
4. 块表达式 - 使用标签块进行复杂初始化
5. 编译时求值 - 顶层常量在运行时之前计算

第三章：类型

Zig 是一种静态类型语言，意味着每个值都有一个在编译时已知的类型。理解类型是编写正确 Zig 代码的基础。

类型系统概览

```
┌──────────────────────────────────────────────────────────────┐
│                        Zig 类型层次                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│                          所有类型                            │
│                             │                                │
│                ┌────────────┴────────────┐                   │
│                │                         │                   │
│        基本类型                          复合类型            │
│            │                             │                   │
│      ┌─────┴─────┐                  ┌──────┴──────┐          │
│      │           │                  │             │          │
│   数值         其他              数组        结构体      联合  │
│     │           │                 │           │          │   │
│   ┌─┴─┐       ┌─┴─┐             切片        枚举       可选   │
│   │   │       │   │             指针                         │
│ 整数 浮点    布尔  Void          向量                         │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

整数类型

Zig 提供对整数大小的细粒度控制：

```
┌──────────────────────────────────────────────────────────────┐
│                        整数类型                              │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   有符号（可以为负）                 无符号（只能为正）        │
│   ──────────────────────            ──────────────────────   │
│   i8   : -128 到 127                u8   : 0 到 255          │
│   i16  : -32768 到 32767            u16  : 0 到 65535        │
│   i32  : -21亿 到 21亿              u32  : 0 到 42亿         │
│   i64  : 巨大范围                   u64  : 0 到巨大          │
│   i128 : 更巨大                     u128 : 更巨大            │
│                                                              │
│   特殊类型：                                                 │
│   ────────────────                                           │
│   isize : 有符号指针大小（32 或 64 位）                      │
│   usize : 无符号指针大小（用于数组索引）                     │
│                                                              │
│   任意宽度：                                                 │
│   ────────────────                                           │
│   u1, u2, u3, ..., u65535   （任意位宽！）                   │
```

```
│    i1, i2, i3, ..., i65535                                          │
│                                                                     │
└─────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 有符号整数（可以为负）
    // ========================================

    const tiny: i8 = -128;        // 8 位：-128 到 127
    const small: i16 = -32768;    // 16 位
    const medium: i32 = -2147483648; // 32 位
    const large: i64 = -9223372036854775808; // 64 位

    std.debug.print("i8 min:  {d}\n", .{tiny});
    std.debug.print("i16 min: {d}\n", .{small});
    std.debug.print("i32 min: {d}\n", .{medium});
    std.debug.print("i64 min: {d}\n", .{large});

    // ========================================
    // 无符号整数（只能为正）
    // ========================================

    const byte: u8 = 255;          // 8 位：0 到 255
    const word: u16 = 65535;       // 16 位
    const dword: u32 = 4294967295; // 32 位
    const qword: u64 = 18446744073709551615; // 64 位

    std.debug.print("\nu8 max:  {d}\n", .{byte});
    std.debug.print("u16 max: {d}\n", .{word});
    std.debug.print("u32 max: {d}\n", .{dword});
    std.debug.print("u64 max: {d}\n", .{qword});

    // ========================================
    // 任意位宽
    // ========================================

    const nibble: u4 = 15;         // 4 位：0 到 15
    const six_bits: u6 = 63;       // 6 位：0 到 63
    const twelve: u12 = 4095;      // 12 位：0 到 4095

    std.debug.print("\nu4 max:  {d}\n", .{nibble});
    std.debug.print("u6 max:  {d}\n", .{six_bits});
    std.debug.print("u12 max: {d}\n", .{twelve});
}
```

输出：

```
i8 min:  -128
i16 min: -32768
i32 min: -2147483648
i64 min: -9223372036854775808
```

```
u8 max:  255
u16 max: 65535
u32 max: 4294967295
u64 max: 18446744073709551615

u4 max:  15
u6 max:  63
u12 max: 4095
```

浮点类型

```
┌──────────────────────────────────────────────────────────────────────┐
│                              浮点类型                                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   类型     大小      精度          范围（约）                         │
│   ────     ────      ──────────    ──────────────                     │
│   f16      16位      ~3 位数       ±65504                            │
│   f32      32位      ~7 位数       ±3.4 × 10^38                      │
│   f64      64位      ~15 位数      ±1.8 × 10^308                     │
│   f128     128位     ~33 位数      ±1.2 × 10^4932                    │
│                                                                      │
│   内存布局（f32 - IEEE 754）：                                        │
│   ┌──────┬──────────┬──────────────────┐                            │
│   │ S    │ 指数     │ 尾数             │                            │
│   │ 1    │ 8 位     │ 23 位            │                            │
│   └──────┴──────────┴──────────────────┘                            │
│                                                                      │
│   例子：f32 中的 3.14159                                              │
│   符号：0（正）                                                       │
│   指数：10000000 (1)                                                  │
│   尾数：10010010000111111010000                                       │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 浮点数
    // ========================================

    const pi_32: f32 = 3.14159265358979;  // 精度较低
    const pi_64: f64 = 3.14159265358979;  // 精度较高

    std.debug.print("f32 pi: {d:.10}\n", .{pi_32});
    std.debug.print("f64 pi: {d:.14}\n", .{pi_64});

    // 科学计数法
    const avogadro: f64 = 6.022e23;  // 6.022 × 10^23
    const planck: f64 = 6.626e-34;  // 6.626 × 10^-34
```

```
    std.debug.print("Avogadro: {e}\n", .{avogadro});
    std.debug.print("Planck: {e}\n", .{planck});

    // 特殊值
    const inf = std.math.inf(f64);      // 无穷大
    const neg_inf = -std.math.inf(f64); // 负无穷大
    const nan = std.math.nan(f64);      // 非数字

    std.debug.print("Infinity: {d}\n", .{inf});
    std.debug.print("NaN: {d}\n", .{nan});

    // 精度很重要！
    const a: f32 = 0.1;
    const b: f32 = 0.2;
    const sum: f32 = a + b;
    std.debug.print("\n0.1 + 0.2 = {d:.20}\n", .{sum});
    std.debug.print("(由于二进制表示，不完全是 0.3)\n", .{});
}
```

输出：

```
f32 pi: 3.1415927410
f64 pi: 3.14159265358979
Avogadro: 6.022e23
Planck: 6.626e-34
Infinity: inf
NaN: nan

0.1 + 0.2 = 0.30000001192092895508
(由于二进制表示，不完全是 0.3)
```

布尔类型

```
const std = @import("std");

pub fn main() void {
    // ========================================
    // 布尔 - true 或 false
    // ========================================

    const is_active: bool = true;
    const is_empty: bool = false;

    std.debug.print("is_active: {}\n", .{is_active});
    std.debug.print("is_empty: {}\n", .{is_empty});

    // 来自比较的布尔值
    const x: i32 = 10;
    const y: i32 = 20;

    const is_equal = x == y;     // false
    const is_less = x < y;       // true
    const is_greater = x > y;    // false
```

```zig
    std.debug.print("\n{d} == {d}: {}\n", .{ x, y, is_equal });
    std.debug.print("{d} < {d}: {}\n", .{ x, y, is_less });
    std.debug.print("{d} > {d}: {}\n", .{ x, y, is_greater });

    // 内存中布尔值是 1 字节
    std.debug.print("\nSize of bool: {d} byte\n", .{@sizeOf(bool)});
}
```

输出：

```
is_active: true
is_empty: false

10 == 20: false
10 < 20: true
10 > 20: false

Size of bool: 1 byte
```

类型转换

```
┌──────────────────────────────────────────────────────────────────┐
│                          类型转换                                 │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   扩展转换（安全 - 自动）                                          │
│   ────────────────────────                                       │
│   u8 ──────► u16 ──────► u32 ──────► u64                         │
│        OK         OK          OK                                 │
│                                                                  │
│   i8 ──────► i16 ──────► i32 ──────► i64                         │
│        OK         OK          OK                                 │
│                                                                  │
│   缩窄转换（危险 - 需要 @intCast）                                 │
│   ──────────────────────────────────────                         │
│   u32 ──────► u16 ──────► u8                                     │
│        ⚠          ⚠         可能丢失数据！                        │
│                                                                  │
│   转换函数：                                                       │
│   ────────────────────                                           │
│   @intCast      : 整数类型之间转换                                 │
│   @floatCast    : 浮点类型之间转换                                 │
│   @intFromFloat : 浮点到整数（截断）                               │
│   @floatFromInt : 整数到浮点                                       │
│   @truncate     : 强制截断                                         │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
```

```zig
    // 扩展转换（安全，隐式）
    // ========================================

    const small: u8 = 42;
    const medium: u16 = small;   // u8 适合 u16，OK
    const large: u32 = medium;   // u16 适合 u32，OK

    std.debug.print("Widening: {d} -> {d} -> {d}\n", .{ small, medium, large });

    // ========================================
    // 缩窄转换（危险，需要 @intCast）
    // ========================================

    const big: u32 = 200;
    const tiny: u8 = @intCast(big);  // 你必须确保它能装下！

    std.debug.print("Narrowing: {d} -> {d}\n", .{ big, tiny });

    // 警告：如果 big > 255 这会崩溃
    // const too_big: u32 = 300;
    // const crash: u8 = @intCast(too_big);  // 运行时错误！

    // ========================================
    // 浮点转换
    // ========================================

    const float64: f64 = 3.14159265358979;
    const float32: f32 = @floatCast(float64);  // 丢失精度

    std.debug.print("\nf64: {d:.15}\n", .{float64});
    std.debug.print("f32: {d:.15}\n", .{float32});

    // 浮点到整数（向零截断）
    const pi: f64 = 3.99999;
    const pi_int: i32 = @intFromFloat(pi);  // 变成 3

    std.debug.print("\nFloat {d:.5} to int: {d}\n", .{ pi, pi_int });

    // 整数到浮点
    const count: i32 = 42;
    const count_float: f64 = @floatFromInt(count);

    std.debug.print("Int {d} to float: {d}\n", .{ count, count_float });

    // ========================================
    // 有符号/无符号转换
    // ========================================

    const positive: u32 = 100;
    const signed: i32 = @intCast(positive);  // 如果值适合则安全

    std.debug.print("\nu32 {d} to i32: {d}\n", .{ positive, signed });
}
```

输出：

```
Widening: 42 -> 42 -> 42
Narrowing: 200 -> 200

f64: 3.141592653589790
f32: 3.141592741012573

Float 3.99999 to int: 3
Int 42 to float: 42

u32 100 to i32: 100
```

使用 @typeInfo 的类型信息

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // 类型内省
    // ========================================

    // 获取类型名称
    std.debug.print("Type names:\n", .{});
    std.debug.print("  i32: {s}\n", .{@typeName(i32)});
    std.debug.print("  []const u8: {s}\n", .{@typeName([]const u8)});
    std.debug.print("  bool: {s}\n", .{@typeName(bool)});

    // 获取类型大小
    std.debug.print("\nType sizes:\n", .{});
    std.debug.print("  i8:   {d} bytes\n", .{@sizeOf(i8)});
    std.debug.print("  i32:  {d} bytes\n", .{@sizeOf(i32)});
    std.debug.print("  i64:  {d} bytes\n", .{@sizeOf(i64)});
    std.debug.print("  f32:  {d} bytes\n", .{@sizeOf(f32)});
    std.debug.print("  f64:  {d} bytes\n", .{@sizeOf(f64)});
    std.debug.print("  bool: {d} byte\n", .{@sizeOf(bool)});

    // 获取对齐
    std.debug.print("\nType alignments:\n", .{});
    std.debug.print("  i32: {d}-byte aligned\n", .{@alignOf(i32)});
    std.debug.print("  i64: {d}-byte aligned\n", .{@alignOf(i64)});
}
```

输出：

```
Type names:
  i32: i32
  []const u8: []const u8
  bool: bool

Type sizes:
  i8:   1 bytes
  i32:  4 bytes
  i64:  8 bytes
  f32:  4 bytes
```

```
  f64:  8 bytes
  bool: 1 byte

Type alignments:
  i32: 4-byte aligned
  i64: 8-byte aligned
```

要点

1. 选择合适的大小 - 当 i8 足够时不要使用 i64
2. 有符号 **vs** 无符号 - 计数/索引使用无符号，算术使用有符号
3. 浮点精度 - f64 更精确但使用更多内存
4. 显式转换 - Zig 要求你在缩窄时显式转换
5. 编译时类型 - 没有类型的数字字面量是 `comptime_int` 或 `comptime_float`

第四章：运算符

运算符是对值执行操作的符号。Zig 提供了一套完整的算术、比较、逻辑和位运算符。

运算符分类

```
┌──────────────────────────────────────────────────────────────┐
│                          运算符分类                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   算术运算符          比较运算符          逻辑运算符         │
│   ────────────        ────────────        ────────           │
│   +   加法            ==   等于           and   逻辑与       │
│   -   减法            !=   不等于         or    逻辑或       │
│   *   乘法            <    小于           !     逻辑非       │
│   /   除法            >    大于                              │
│   %   取模            <=   小于等于                          │
│   -   取负            >=   大于等于                          │
│                                                              │
│   位运算符            赋值运算符          特殊说明           │
│   ────────            ────────────        ────────           │
│   &    与             =    赋值           ++  (Zig 中没有！) │
│   |    或             +=   加法赋值       --  (Zig 中没有！) │
│   ^    异或           -=   减法赋值       ?:  (使用 if-else) │
│   ~    取反           *=   乘法赋值                          │
│   <<   左移           /=   除法赋值                          │
│   >>   右移           %=   取模赋值                          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

算术运算符

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 算术运算符 ===\n\n", .{});

    const a: i32 = 17;
    const b: i32 = 5;

    // 基本运算
    std.debug.print("{d} + {d} = {d}\n", .{ a, b, a + b });   // 22
    std.debug.print("{d} - {d} = {d}\n", .{ a, b, a - b });   // 12
    std.debug.print("{d} * {d} = {d}\n", .{ a, b, a * b });   // 85

    // Zig 中的除法需要显式函数
    // @divTrunc - 向零截断
    // @divFloor - 向负无穷取整
    std.debug.print("{d} / {d} = {d} (截断)\n", .{ a, b, @divTrunc(a, b) });

    // 取模
    std.debug.print("{d} %% {d} = {d}\n", .{ a, b, @mod(a, b) });

    // 取负
    std.debug.print("-{d} = {d}\n", .{ a, -a });

    // ========================================
    // 溢出安全运算符
    // ========================================

    std.debug.print("\n=== 溢出处理 ===\n\n", .{});

    // 回绕运算符（溢出时回绕）
    var x: u8 = 250;
    x +%= 10;  // 回绕：250 + 10 = 260 -> 4 (260 % 256)
    std.debug.print("250 +%% 10 = {d} (回绕)\n", .{x});

    // 饱和运算符（在最大/最小值处截止）
    var y: u8 = 250;
    y +|= 10;  // 饱和：250 + 10 = 255 (u8 最大值)
    std.debug.print("250 +| 10 = {d} (饱和)\n", .{y});
}
```

输出：

```
=== 算术运算符 ===

17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 (截断)
17 % 5 = 2
-17 = -17

=== 溢出处理 ===
```

```
250 +% 10 = 4 (回绕)
250 +| 10 = 255 (饱和)
```

溢出处理可视化

```
┌──────────────────────────────────────────────────────────────────────┐
│                              溢出处理                                  │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   u8 范围：0 ──────────────────────────────────────────► 255         │
│                                                                      │
│   普通加法 (+)：250 + 10 = ?                                          │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  调试模式：恐慌！检测到溢出！                                │       │
│   │  发布模式：未定义行为（糟糕！）                              │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│   回绕加法 (+%)：250 +% 10 = 4                                        │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  250 + 10 = 260                                          │       │
│   │  260 mod 256 = 4                                         │       │
│   │  就像时钟一样回绕                                            │       │
│   │                                                          │       │
│   │  0 ←──────────────────────────────────── 255             │       │
│   │  ↑                                        │              │       │
│   │  └──────────────── 回绕 ──────────────────┘              │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                                                      │
│   饱和加法 (+|)：250 +| 10 = 255                                      │
│   ┌──────────────────────────────────────────────────────────┐       │
│   │  250 + 10 = 260                                          │       │
│   │  260 > 255，所以截止到 255                                   │       │
│   │  就像一个不会溢出的杯子                                      │       │
│   │                                                          │       │
│   │  结果：═══════════════════════════════╣ 255               │       │
│   │          （停在最大值）                                      │       │
│   └──────────────────────────────────────────────────────────┘       │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

比较运算符

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 比较运算符 ===\n\n", .{});

    const x: i32 = 10;
    const y: i32 = 20;
    const z: i32 = 10;
```

```zig
    // 相等性
    std.debug.print("{d} == {d}: {}\n", .{ x, y, x == y });   // false
    std.debug.print("{d} == {d}: {}\n", .{ x, z, x == z });   // true
    std.debug.print("{d} != {d}: {}\n", .{ x, y, x != y });   // true

    // 关系运算
    std.debug.print("\n{d} <  {d}: {}\n", .{ x, y, x < y });  // true
    std.debug.print("{d} >  {d}: {}\n", .{ x, y, x > y });    // false
    std.debug.print("{d} <= {d}: {}\n", .{ x, z, x <= z });   // true
    std.debug.print("{d} >= {d}: {}\n", .{ x, y, x >= y });   // false

    // 比较结果是布尔值
    const is_equal: bool = x == z;
    const is_greater: bool = x > y;

    std.debug.print("\n存储的结果：is_equal={}, is_greater={}\n", .{ is_equal, is_greater });
}
```

输出：

```
=== 比较运算符 ===

10 == 20: false
10 == 10: true
10 != 20: true

10 <  20: true
10 >  20: false
10 <= 10: true
10 >= 20: false

存储的结果：is_equal=true, is_greater=false
```

逻辑运算符

```
┌──────────────────────────────────────────────────────────────┐
│                          逻辑运算符                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   AND (and)：两者都必须为真                                  │
│   ┌──────────────────────────────────────────┐               │
│   │  A      │  B      │  A and B              │               │
│   ├─────────┼─────────┼───────────────────────┤               │
│   │ false   │ false   │   false               │               │
│   │ false   │ true    │   false               │               │
│   │ true    │ false   │   false               │               │
│   │ true    │ true    │   true   ✓            │               │
│   └──────────────────────────────────────────┘               │
│                                                              │
│   OR (or)：至少一个必须为真                                  │
│   ┌──────────────────────────────────────────┐               │
│   │  A      │  B      │  A or B               │               │
```

```
│    ├─────────┼─────────┼──────────────────────────────┤          │
│    │ false   │ false   │   false                      │          │
│    │ false   │ true    │   true   ✓                   │          │
│    │ true    │ false   │   true   ✓                   │          │
│    │ true    │ true    │   true   ✓                   │          │
│    └─────────────────────────────────────────────────┘          │
│                                                                  │
│   NOT (!) : 反转值                                               │
│    ┌─────────────────────────────────────────────────┐          │
│    │  A      │  !A                                   │          │
│    ├─────────┼───────────────────────────────────────┤          │
│    │ false   │  true                                 │          │
│    │ true    │  false                                │          │
│    └─────────────────────────────────────────────────┘          │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 逻辑运算符 ===\n\n", .{});

    const t = true;
    const f = false;

    // AND - 两者都必须为真
    std.debug.print("true  and true  = {}\n", .{t and t});
    std.debug.print("true  and false = {}\n", .{t and f});
    std.debug.print("false and true  = {}\n", .{f and t});
    std.debug.print("false and false = {}\n", .{f and f});

    // OR - 至少一个必须为真
    std.debug.print("\ntrue  or true  = {}\n", .{t or t});
    std.debug.print("true  or false = {}\n", .{t or f});
    std.debug.print("false or true  = {}\n", .{f or t});
    std.debug.print("false or false = {}\n", .{f or f});

    // NOT - 取反
    std.debug.print("\nnot true  = {}\n", .{!t});
    std.debug.print("not false = {}\n", .{!f});

    // ========================================
    // 短路求值
    // ========================================

    std.debug.print("\n=== 短路求值 ===\n\n", .{});

    // 使用 'and' 时，如果第一个为 false，则不评估第二个
    // 使用 'or' 时，如果第一个为 true，则不评估第二个

    const x: i32 = 0;

    // 使用短路进行安全的除法检查
```

```zig
    const safe = x != 0 and @divTrunc(10, x) > 0;
    std.debug.print(" 除法检查 (x=0): {}\n", .{safe});
    // 因为 x != 0 为 false，所以除法永远不会发生！
}
```

输出：

```
=== 逻辑运算符 ===

true  and true  = true
true  and false = false
false and true  = false
false and false = false

true  or true  = true
true  or false = true
false or true  = true
false or false = false

not true  = false
not false = true

=== 短路求值 ===

除法检查 (x=0): false
```

位运算符

```
┌──────────────────────────────────────────────────────────────┐
│                          位运算操作                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   给定：A = 0b11001010  (202)                                │
│         B = 0b10101100  (172)                                │
│                                                              │
│   AND (&)：两个位都必须为 1                                  │
│   ┌──────────────────────────────────────────┐               │
│   │  A:     1 1 0 0 1 0 1 0                  │               │
│   │  B:     1 0 1 0 1 1 0 0                  │               │
│   │         ─────────────────                │               │
│   │  A & B: 1 0 0 0 1 0 0 0  = 136           │               │
│   └──────────────────────────────────────────┘               │
│                                                              │
│   OR (|)：至少一个位必须为 1                                 │
│   ┌──────────────────────────────────────────┐               │
│   │  A:     1 1 0 0 1 0 1 0                  │               │
│   │  B:     1 0 1 0 1 1 0 0                  │               │
│   │         ─────────────────                │               │
│   │  A | B: 1 1 1 0 1 1 1 0  = 238           │               │
│   └──────────────────────────────────────────┘               │
│                                                              │
│   XOR (^)：恰好一个位必须为 1                                │
```

```
A:     1 1 0 0 1 0 1 0
B:     1 0 1 0 1 1 0 0
       ─────────────────
A ^ B: 0 1 1 0 0 1 1 0  = 102
```

NOT (~)：翻转所有位

```
A:     1 1 0 0 1 0 1 0
       ─────────────────
~A:    0 0 1 1 0 1 0 1  = 53
```

左移 (<<)：位向左移动，用 0 填充

```
X:      0 0 0 0 1 1 1 1  = 15
X << 2: 0 0 1 1 1 1 0 0  = 60
        (乘以 2^n)
```

右移 (>>)：位向右移动

```
X:      0 0 0 0 1 1 1 1  = 15
X >> 2: 0 0 0 0 0 0 1 1  = 3
        (除以 2^n)
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 位运算符 ===\n\n", .{});

    const a: u8 = 0b11001010;  // 202
    const b: u8 = 0b10101100;  // 172

    std.debug.print("a     = 0b{b:0>8} ({d})\n", .{ a, a });
    std.debug.print("b     = 0b{b:0>8} ({d})\n", .{ b, b });

    std.debug.print("\na & b = 0b{b:0>8} ({d}) - AND\n", .{ a & b, a & b });
    std.debug.print("a | b = 0b{b:0>8} ({d}) - OR\n", .{ a | b, a | b });
    std.debug.print("a ^ b = 0b{b:0>8} ({d}) - XOR\n", .{ a ^ b, a ^ b });
    std.debug.print("~a    = 0b{b:0>8} ({d}) - NOT\n", .{ ~a, ~a });

    // 位移
    const x: u8 = 0b00001111;  // 15
    std.debug.print("\nx      = 0b{b:0>8} ({d})\n", .{ x, x });
    std.debug.print("x << 2 = 0b{b:0>8} ({d}) - 左移\n", .{ x << 2, x << 2 });
    std.debug.print("x >> 2 = 0b{b:0>8} ({d}) - 右移\n", .{ x >> 2, x >> 2 });
```

```zig
    // 实际示例：提取位
    std.debug.print("\n=== 实际应用：位标志 ===\n\n", .{});

    const READ: u8 = 0b00000001;   // 位 0
    const WRITE: u8 = 0b00000010;  // 位 1
    const EXECUTE: u8 = 0b00000100; // 位 2

    var permissions: u8 = 0;

    // 设置权限
    permissions |= READ;      // 添加读取权限
    permissions |= WRITE;     // 添加写入权限
    std.debug.print(" 添加 READ|WRITE 后: 0b{b:0>8}\n", .{permissions});

    // 检查权限
    const can_read = (permissions & READ) != 0;
    const can_exec = (permissions & EXECUTE) != 0;
    std.debug.print(" 可以读取: {}, 可以执行: {}\n", .{ can_read, can_exec });

    // 移除权限
    permissions &= ~WRITE;    // 移除写入权限
    std.debug.print(" 移除 WRITE 后: 0b{b:0>8}\n", .{permissions});
}
```

输出：

```
=== 位运算符 ===

a     = 0b11001010 (202)
b     = 0b10101100 (172)

a & b = 0b10001000 (136) - AND
a | b = 0b11101110 (238) - OR
a ^ b = 0b01100110 (102) - XOR
~a    = 0b00110101 (53) - NOT

x      = 0b00001111 (15)
x << 2 = 0b00111100 (60) - 左移
x >> 2 = 0b00000011 (3) - 右移

=== 实际应用：位标志 ===

添加 READ|WRITE 后: 0b00000011
可以读取: true, 可以执行: false
移除 WRITE 后: 0b00000001
```

要点

1. 没有 ++ 或–运算符 - 改用 += 1 和 –= 1
2. 显式溢出处理 - 使用 +%、–%、*% 进行回绕，使用 +|、–|、*| 进行饱和
3. 短路求值 - and 和 or 在不需要时不会评估第二个操作数
4. 位运算符 - 对于标志、掩码和底层编程非常强大
5. 除法函数 - 使用 @divTrunc、@divFloor、@mod、@rem 获得明确的行为

第五章：数组

数组是固定大小、连续存储的同类型元素集合。它们是 Zig 中最基本的数据结构之一。

数组内存布局

```
┌──────────────────────────────────────────────────────────────────┐
│                          内存中的数组                                │
├──────────────────────────────────────────────────────────────────┤
│                                                                    │
│   声明：const arr: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };          │
│                                                                    │
│   内存布局（假设 i32 为 4 字节）：                                    │
│                                                                    │
│   地址：    0x100   0x104   0x108   0x10C   0x110                  │
│                                                                    │
│   值：      ┌──────┬──────┬──────┬──────┬──────┐                  │
│             │  10  │  20  │  30  │  40  │  50  │                  │
│             └──────┴──────┴──────┴──────┴──────┘                  │
│   索引：       [0]    [1]    [2]    [3]    [4]                      │
│                                                                    │
│   关键属性：                                                         │
│   • 大小在编译时固定                                                 │
│   • 元素是连续的（没有间隙）                                         │
│   • 按索引访问是 O(1)                                                │
│   • 总大小 = 元素大小 × 数量 = 4 × 5 = 20 字节                       │
│                                                                    │
└──────────────────────────────────────────────────────────────────┘
```

创建数组

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 创建数组 ===\n\n", .{});

    // ========================================
    // 方法 1：显式类型和值
    // ========================================
    const explicit: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };
    std.debug.print(" 显式声明: {any}\n", .{explicit});

    // ========================================
    // 方法 2：使用 [_] 推断长度
    // ========================================
    const inferred = [_]i32{ 1, 2, 3, 4 };  // 编译器计数：4 个元素
    std.debug.print(" 推断长度: {any}\n", .{inferred});
    std.debug.print(" 长度: {d}\n", .{inferred.len});
```

```
    // ========================================
    // 方法 3：用相同的值填充
    // ========================================
    const zeros = [_]i32{0} ** 5;    // [0, 0, 0, 0, 0]
    const ones = [_]i32{1} ** 3;     // [1, 1, 1]
    const magic = [_]i32{42} ** 4;   // [42, 42, 42, 42]

    std.debug.print(" 零数组: {any}\n", .{zeros});
    std.debug.print(" 一数组: {any}\n", .{ones});
    std.debug.print(" 魔数: {any}\n", .{magic});

    // ========================================
    // 方法 4：重复模式
    // ========================================
    const pattern = [_]i32{ 1, 2 } ** 3;  // [1, 2, 1, 2, 1, 2]
    const wave = [_]i32{ 1, 2, 3, 2 } ** 2;

    std.debug.print(" 模式: {any}\n", .{pattern});
    std.debug.print(" 波形: {any}\n", .{wave});

    // ========================================
    // 方法 5：字符串数组
    // ========================================
    const names = [_][]const u8{ "Alice", "Bob", "Charlie" };
    std.debug.print(" 名字: ", .{});
    for (names) |name| {
        std.debug.print("{s} ", .{name});
    }
    std.debug.print("\n", .{});
}
```

输出：

```
=== 创建数组 ===

显式声明: { 10, 20, 30, 40, 50 }
推断长度: { 1, 2, 3, 4 }
长度: 4
零数组: { 0, 0, 0, 0, 0 }
一数组: { 1, 1, 1 }
魔数: { 42, 42, 42, 42 }
模式: { 1, 2, 1, 2, 1, 2 }
波形: { 1, 2, 3, 2, 1, 2, 3, 2 }
名字: Alice Bob Charlie
```

访问和修改数组

```
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 访问数组 ===\n\n", .{});

    const numbers = [_]i32{ 100, 200, 300, 400, 500 };
```

```zig
    // 按索引访问
    std.debug.print(" 第一个元素 (索引 0): {d}\n", .{numbers[0]});
    std.debug.print(" 第三个元素 (索引 2): {d}\n", .{numbers[2]});
    std.debug.print(" 最后一个元素 (索引 {}): {d}\n", .{ numbers.len - 1, numbers[numbers.len - 1

    // ========================================
    // 修改数组(必须是 var)
    // ========================================

    std.debug.print("\n=== 修改数组 ===\n\n", .{});

    var mutable = [_]i32{ 1, 2, 3, 4, 5 };
    std.debug.print(" 修改前: {any}\n", .{mutable});

    // 修改单个元素
    mutable[0] = 100;
    mutable[4] = 500;
    std.debug.print(" 修改 mutable[0]=100, [4]=500 后: {any}\n", .{mutable});

    // 在循环中修改所有元素
    for (&mutable) |*element| {
        element.* *= 2;
    }
    std.debug.print(" 全部翻倍后: {any}\n", .{mutable});
}
```

输出:

```
=== 访问数组 ===

第一个元素 (索引 0): 100
第三个元素 (索引 2): 300
最后一个元素 (索引 4): 500

=== 修改数组 ===

修改前: { 1, 2, 3, 4, 5 }
修改 mutable[0]=100, [4]=500 后: { 100, 2, 3, 4, 500 }
全部翻倍后: { 200, 4, 6, 8, 1000 }
```

遍历数组

```
┌──────────────────────────────────────────────────────────┐
│                      数组遍历方法                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   1. 简单遍历(仅值):                                      │
│      for (array) |value| { ... }                         │
│                                                          │
│   2. 带索引:                                              │
│      for (array, 0..) |value, index| { ... }             │
│                                                          │
│   3. 带指针(用于修改):                                    │
```

```
│      for (&array) |*ptr| { ptr.* = newValue; }                │
│                                                                │
│   4. 多个数组（zip）：                                          │
│      for (arr1, arr2) |a, b| { ... }                           │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 数组遍历 ===\n\n", .{});

    const numbers = [_]i32{ 10, 20, 30, 40, 50 };

    // ========================================
    // 方法 1：仅值
    // ========================================
    std.debug.print(" 值: ", .{});
    for (numbers) |value| {
        std.debug.print("{d} ", .{value});
    }
    std.debug.print("\n", .{});

    // ========================================
    // 方法 2：带索引
    // ========================================
    std.debug.print("\n带索引:\n", .{});
    for (numbers, 0..) |value, index| {
        std.debug.print("  [{d}] = {d}\n", .{ index, value });
    }

    // ========================================
    // 方法 3：带指针（修改）
    // ========================================
    var mutable = [_]i32{ 1, 2, 3, 4, 5 };

    for (&mutable) |*ptr| {
        ptr.* *= 10;  // 每个乘以 10
    }
    std.debug.print("\n乘 10 后: {any}\n", .{mutable});

    // ========================================
    // 方法 4：多个数组（zip）
    // ========================================
    const a = [_]i32{ 1, 2, 3 };
    const b = [_]i32{ 10, 20, 30 };

    std.debug.print("\nZip 遍历:\n", .{});
    for (a, b, 0..) |x, y, i| {
        std.debug.print("  [{d}] a={d}, b={d}, sum={d}\n", .{ i, x, y, x + y });
    }
}
```

输出：

```
=== 数组遍历 ===

值: 10 20 30 40 50

带索引:
  [0] = 10
  [1] = 20
  [2] = 30
  [3] = 40
  [4] = 50

乘 10 后: { 10, 20, 30, 40, 50 }

Zip 遍历:
  [0] a=1, b=10, sum=11
  [1] a=2, b=20, sum=22
  [2] a=3, b=30, sum=33
```

多维数组

```
┌──────────────────────────────────────────────────────────────────────┐
│                          二维数组（矩阵）布局                          │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   声明：const matrix: [3][4]i32 = ...                                │
│                                                                      │
│   逻辑视图（3 行，4 列）：                                            │
│                                                                      │
│           第 0 列   第 1 列   第 2 列   第 3 列                       │
│         ┌─────────┬─────────┬─────────┬─────────┐                   │
│   第 0 行 │    1    │    2    │    3    │    4    │                   │
│         ├─────────┼─────────┼─────────┼─────────┤                   │
│   第 1 行 │    5    │    6    │    7    │    8    │                   │
│         ├─────────┼─────────┼─────────┼─────────┤                   │
│   第 2 行 │    9    │   10    │   11    │   12    │                   │
│         └─────────┴─────────┴─────────┴─────────┘                   │
│                                                                      │
│   访问：matrix[行][列]                                               │
│   示例：matrix[1][2] = 7                                             │
│                                                                      │
│   内存布局（行优先，连续）：                                          │
│   ┌──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┬──┐                            │
│   │1 │2 │3 │4 │5 │6 │7 │8 │9 │10│11│12│                            │
│   └──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┴──┘                            │
│   第 0 行     第 1 行       第 2 行                                    │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 多维数组 ===\n\n", .{});
```

```zig
    // ========================================
    // 二维数组（矩阵）
    // ========================================
    const matrix = [3][4]i32{
        [_]i32{ 1, 2, 3, 4 },
        [_]i32{ 5, 6, 7, 8 },
        [_]i32{ 9, 10, 11, 12 },
    };

    // 访问单个元素
    std.debug.print("matrix[1][2] = {d}\n", .{matrix[1][2]});  // 7

    // 打印整个矩阵
    std.debug.print("\n矩阵:\n", .{});
    for (matrix, 0..) |row, i| {
        std.debug.print("  第 {d} 行: ", .{i});
        for (row) |value| {
            std.debug.print("{d:4}", .{value});
        }
        std.debug.print("\n", .{});
    }

    // ========================================
    // 三维数组
    // ========================================
    const cube = [2][2][2]i32{
        [_][2]i32{
            [_]i32{ 1, 2 },
            [_]i32{ 3, 4 },
        },
        [_][2]i32{
            [_]i32{ 5, 6 },
            [_]i32{ 7, 8 },
        },
    };

    std.debug.print("\ncube[1][0][1] = {d}\n", .{cube[1][0][1]});  // 6
}
```

输出：

```
=== 多维数组 ===

matrix[1][2] = 7

矩阵:
  第 0 行:    1   2   3   4
  第 1 行:    5   6   7   8
  第 2 行:    9  10  11  12

cube[1][0][1] = 6
```

数组操作

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 数组操作 ===\n\n", .{});

    // ========================================
    // 连接（仅编译时）
    // ========================================
    const a = [_]i32{ 1, 2, 3 };
    const b = [_]i32{ 4, 5 };
    const combined = a ++ b;  // [1, 2, 3, 4, 5]

    std.debug.print(" 连接: {any} ++ {any} = {any}\n", .{ a, b, combined });

    // ========================================
    // 乘法（仅编译时）
    // ========================================
    const pattern = [_]i32{ 1, 2 };
    const repeated = pattern ** 4;  // [1, 2, 1, 2, 1, 2, 1, 2]

    std.debug.print(" 重复: {any} ** 4 = {any}\n", .{ pattern, repeated });

    // ========================================
    // 数组大小信息
    // ========================================
    const arr: [5]i32 = [_]i32{ 10, 20, 30, 40, 50 };

    std.debug.print("\n数组信息:\n", .{});
    std.debug.print("  长度: {d} 个元素\n", .{arr.len});
    std.debug.print("  元素大小: {d} 字节\n", .{@sizeOf(i32)});
    std.debug.print("  总大小: {d} 字节\n", .{@sizeOf(@TypeOf(arr))});
}
```

输出：

```
=== 数组操作 ===

连接: { 1, 2, 3 } ++ { 4, 5 } = { 1, 2, 3, 4, 5 }
重复: { 1, 2 } ** 4 = { 1, 2, 1, 2, 1, 2, 1, 2 }

数组信息:
  长度: 5 个元素
  元素大小: 4 字节
  总大小: 20 字节
```

要点

1. 固定大小 - 数组长度在编译时已知且不能改变
2. 从零开始索引 - 第一个元素在索引 0
3. 边界检查 - Zig 在调试模式下检查数组边界
4. 连续内存 - 元素连续存储
5. 使用 `[_]` 推断长度 - 编译器为你计数元素

6. ++ 和 ** 仅限编译时 - 运行时不能连接数组

第六章：切片

切片是对数组或其他连续内存的视图。与数组不同，切片不拥有数据——它们只是引用数据。这使得它们在处理数组的部分或将数据传递给函数时非常灵活。

切片 vs 数组

```
┌──────────────────────────────────────────────────────────────────┐
│                          数组 vs 切片                            │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   数组：[5]i32                                                   │
│   • 固定大小（5 个元素）                                         │
│   • 大小在编译时已知                                             │
│   • 拥有其数据                                                   │
│   • 内联存储                                                     │
│                                                                  │
│   ┌─────┬─────┬─────┬─────┬─────┐                               │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │   （内联数据）                │
│   └─────┴─────┴─────┴─────┴─────┘                               │
│                                                                  │
│   切片：[]i32                                                    │
│   • 可变大小                                                     │
│   • 大小在运行时已知                                             │
│   • 引用数据（不拥有它）                                         │
│   • 只是一个指针 + 长度                                          │
│                                                                  │
│   ┌──────────────┐                                               │
│   │ ptr │ len=5  │──────┐                                        │
│   └──────────────┘      │                                        │
│                         ▼                                        │
│   ┌─────┬─────┬─────┬─────┬─────┐                               │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │   （数据在别处）              │
│   └─────┴─────┴─────┴─────┴─────┘                               │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

创建切片

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 创建切片 ===\n\n", .{});

    const array = [_]i32{ 10, 20, 30, 40, 50, 60, 70 };
```

```zig
    // ========================================
    // 方法 1：切片整个数组
    // ========================================
    const all: []const i32 = &array;
    std.debug.print(" 整个数组: {any}\n", .{all});
    std.debug.print(" 长度: {d}\n", .{all.len});

    // ========================================
    // 方法 2：使用范围切片 [start..end)
    // ========================================
    const middle = array[2..5];  // 索引 2, 3, 4 处的元素
    std.debug.print("\nmiddle[2..5]: {any}\n", .{middle});

    // ========================================
    // 方法 3：开放式切片
    // ========================================
    const from_start = array[0..3];   // 前 3 个元素
    const to_end = array[4..];        // 从索引 4 到末尾
    const everything = array[0..];    // 整个数组

    std.debug.print("array[0..3]: {any}\n", .{from_start});
    std.debug.print("array[4..]:  {any}\n", .{to_end});
    std.debug.print("array[0..]: {any}\n", .{everything});
}
```

输出：

```
=== 创建切片 ===

整个数组: { 10, 20, 30, 40, 50, 60, 70 }
长度: 7

middle[2..5]: { 30, 40, 50 }
array[0..3]: { 10, 20, 30 }
array[4..]:  { 50, 60, 70 }
array[0..]: { 10, 20, 30, 40, 50, 60, 70 }
```

切片范围可视化

```
┌──────────────────────────────────────────────────────────────┐
│                          切片范围                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   原始数组：[10, 20, 30, 40, 50, 60, 70]                     │
│   索引：       0   1   2   3   4   5   6                     │
│                                                              │
│   array[2..5]                                                │
│   ┌────┬────┬────┬────┬────┬────┬────┐                      │
│   │ 10 │ 20 │ 30 │ 40 │ 50 │ 60 │ 70 │                      │
│   └────┴────┴────┴────┴────┴────┴────┘                      │
│                  ↑──────────────↑                            │
│                  2              5（不包含）                   │
│   结果：[30, 40, 50]                                         │
```

```
│                                                          │
│   array[0..3]  → [10, 20, 30]                            │
│   array[4..]   → [50, 60, 70]  (4 到末尾)                 │
│   array[..4]   → [10, 20, 30, 40]  (开头到 4)             │
│                                                          │
│   注意：结束索引是不包含的（不被包括）                      │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

切片作为函数参数

这是切片真正闪光的地方——它们让你编写可以处理任意大小数组的函数：

```zig
const std = @import("std");

// 这个函数可以处理任意大小的数组！
fn sum(values: []const i32) i32 {
    var total: i32 = 0;
    for (values) |v| {
        total += v;
    }
    return total;
}

fn printAll(data: []const u8) void {
    std.debug.print(" 数据: {s}\n", .{data});
}

fn average(values: []const i32) f64 {
    if (values.len == 0) return 0;
    const s = sum(values);
    return @as(f64, @floatFromInt(s)) / @as(f64, @floatFromInt(values.len));
}

pub fn main() void {
    std.debug.print("=== 切片作为参数 ===\n\n", .{});

    // 不同大小的数组
    const small = [_]i32{ 1, 2, 3 };
    const large = [_]i32{ 10, 20, 30, 40, 50 };

    // 同一个函数可以处理两者！
    std.debug.print("small 的总和: {d}\n", .{sum(&small)});
    std.debug.print("large 的总和: {d}\n", .{sum(&large)});

    // 甚至可以处理部分切片
    std.debug.print("large[1..4] 的总和: {d}\n", .{sum(large[1..4])});

    // 平均值
    std.debug.print("\nsmall 的平均值: {d:.2}\n", .{average(&small)});
    std.debug.print("large 的平均值: {d:.2}\n", .{average(&large)});

    // 字符串切片
```

```
    printAll("Hello, Zig!");
}
```

输出：

```
=== 切片作为参数 ===

small 的总和: 6
large 的总和: 150
large[1..4] 的总和: 90

small 的平均值: 2.00
large 的平均值: 30.00
数据: Hello, Zig!
```

要点

1. 切片不拥有数据 - 它们是对现有内存的视图
2. 范围语法 **[start..end]** - 结束是不包含的
3. 灵活的函数参数 - 使用 `[]const T` 接受任意大小
4. 运行时长度 - 与数组不同，切片长度可以在运行时计算
5. 内存高效 - 只是一个指针 + 长度（64 位系统上 16 字节）

第七章：指针

指针存储内存地址。它们对于系统编程至关重要，允许你在不复制数据的情况下引用数据。

指针基础

```
┌──────────────────────────────────────────────────────────────┐
│                          指针概念                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   变量 x 在地址 0x1000：                                      │
│                                                              │
│   ┌──────────────┬──────────┬──────────┐                    │
│   │  地址        │  值      │  名称    │                    │
│   ├──────────────┼──────────┼──────────┤                    │
│   │  0x1000      │   42     │   x      │                    │
│   └──────────────┴──────────┴──────────┘                    │
│                                                              │
│   指针 ptr 指向 x：                                           │
│                                                              │
│   ┌──────────────┬──────────┬──────────┐                    │
│   │  地址        │  值      │  名称    │                    │
│   ├──────────────┼──────────┼──────────┤                    │
│   │  0x2000      │ 0x1000   │   ptr    │──────┐             │
│   └──────────────┴──────────┴──────────┘      │             │
│                                               │             │
│                                               │             │
│                                               │             │
```

```
│                                                                │
│                              ▼                                 │
│    ┌──────────────────────────────────────────┐                │
│    │  0x1000      │    42     │    x          │                │
│    └──────────────┴───────────┴───────────────┘                │
│                                                                │
│    ptr.*  表示 "ptr 中存储的地址处的值"                          │
│    &x     表示 "x 的地址"                                       │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 指针基础 ===\n\n", .{});

    // 创建一个变量
    var value: i32 = 42;

    // 用 & 获取指向它的指针
    const ptr: *i32 = &value;

    // 用 .* 通过指针读取
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});

    // 通过指针修改
    ptr.* = 100;
    std.debug.print("\n修改 ptr.* = 100 后:\n", .{});
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});
}
```

输出：

```
=== 指针基础 ===

value = 42
ptr.* = 42

修改 ptr.* = 100 后:
value = 100
ptr.* = 100
```

指针类型

```
┌──────────────────────────────────────────────────────────────┐
│                          指针类型                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    *T          单项指针                                      │
│                指向恰好一个 T                                 │
│                示例：*i32, *bool, *MyStruct                   │
│                                                              │
│    [*]T        多项指针                                      │
```

```
│                 指向未知数量的 T                                  │
│                 没有长度信息                                      │
│                                                                   │
│    []T          切片（指针 + 长度）                               │
│                 带有运行时长度的指针                              │
│                 比 [*]T 更安全                                    │
│                                                                   │
│    *const T     指向常量的指针                                    │
│                 不能通过此指针修改值                              │
│                                                                   │
│    ?*T          可选指针                                          │
│                 可以为 null                                       │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

通过指针传递

```zig
const std = @import("std");

// 接受一个指针 - 可以修改原始值
fn increment(ptr: *i32) void {
    ptr.* += 1;
}

// 接受一个切片 - 可以修改所有元素
fn doubleAll(values: []i32) void {
    for (values) |*v| {
        v.* *= 2;
    }
}

// 使用指针交换两个值
fn swap(a: *i32, b: *i32) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
}

pub fn main() void {
    std.debug.print("=== 通过指针传递 ===\n\n", .{});

    var num: i32 = 5;
    std.debug.print(" 初始值: {d}\n", .{num});

    increment(&num);
    std.debug.print(" 增加后: {d}\n", .{num});

    increment(&num);
    increment(&num);
    std.debug.print(" 再增加 2 次后: {d}\n", .{num});

    // 数组修改
```

```zig
    var arr = [_]i32{ 1, 2, 3, 4, 5 };
    std.debug.print("\n数组修改前: {any}\n", .{arr});
    doubleAll(&arr);
    std.debug.print(" 翻倍后: {any}\n", .{arr});

    // 交换
    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\n交换前: x={d}, y={d}\n", .{ x, y });
    swap(&x, &y);
    std.debug.print(" 交换后: x={d}, y={d}\n", .{ x, y });
}
```

输出：

```
=== 通过指针传递 ===

初始值: 5
增加后: 6
再增加 2 次后: 8

数组修改前: { 1, 2, 3, 4, 5 }
翻倍后: { 2, 4, 6, 8, 10 }

交换前: x=100, y=200
交换后: x=200, y=100
```

要点

1. **&** 获取地址 - 使用 &x 获取 x 的指针
2. **.*** 解引用 - 使用 ptr.* 访问值
3. 指针实现修改 - 通过指针传递以修改原始数据
4. 常量指针 - *const T 防止通过该指针修改
5. 默认没有 **null** - 普通指针不能为 null；使用 ?*T 表示可空

第八章：字符串

在 Zig 中，字符串只是字节数组（[]const u8）。没有特殊的字符串类型——这使语言保持简单并给你完全的控制权。

字符串类型

```
┌──────────────────────────────────────────────────────────┐
│                    Zig 中的字符串类型                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   类型                  描述                             │
│   ────                  ────────────                     │
│   []const u8            字节切片（最常用）               │
│   *const [N:0]u8        指向以 null 结尾字符串的指针     │
│   [:0]const u8          以哨兵终止的切片                 │
```

```
│   [N]u8                固定大小的字节数组                          │
│                                                                    │
│   字符串字面量 "Hello"：                                            │
│                                                                    │
│   内存：│ H │ e │ l │ l │ o │ \0 │                                 │
│         └──────────────────────────┘                               │
│          类型：*const [5:0]u8                                      │
│          强制转换为：[]const u8                                     │
│                                                                    │
│   关键洞察：字符串在 Zig 中不是特殊的。                              │
│   它们只是 u8（字节）数组。                                         │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== 字符串基础 ===\n\n", .{});

    // 字符串字面量 - 类型是 *const [11:0]u8
    // 但通常强制转换为 []const u8
    const greeting: []const u8 = "Hello, Zig!";

    std.debug.print(" 字符串: {s}\n", .{greeting});
    std.debug.print(" 长度: {d}\n", .{greeting.len});

    // 访问单个字节
    std.debug.print(" 第一个字节: '{c}' (ASCII {d})\n", .{ greeting[0], greeting[0] });
    std.debug.print(" 最后一个字节: '{c}'\n", .{greeting[greeting.len - 1]});

    // 遍历字符
    std.debug.print("\n字符: ", .{});
    for (greeting) |char| {
        std.debug.print("{c} ", .{char});
    }
    std.debug.print("\n", .{});
}
```

输出：

```
=== 字符串基础 ===

字符串: Hello, Zig!
长度: 11
第一个字节: 'H' (ASCII 72)
最后一个字节: '!'

字符: H e l l o ,  Z i g !
```

字符串操作

```zig
const std = @import("std");
```

```zig
pub fn main() void {
    std.debug.print("=== 字符串操作 ===\n\n", .{});

    const text = "The quick brown fox jumps";

    // 切片（子字符串）
    const first_word = text[0..3];
    const last_word = text[20..];

    std.debug.print(" 原始: {s}\n", .{text});
    std.debug.print(" 第一个词: {s}\n", .{first_word});
    std.debug.print(" 最后一个词: {s}\n", .{last_word});

    // 比较
    std.debug.print("\n=== 比较 ===\n", .{});
    const a = "apple";
    const b = "apple";
    const c = "banana";

    std.debug.print("'{s}' == '{s}': {}\n", .{ a, b, std.mem.eql(u8, a, b) });
    std.debug.print("'{s}' == '{s}': {}\n", .{ a, c, std.mem.eql(u8, a, c) });

    // 查找子字符串
    std.debug.print("\n=== 搜索 ===\n", .{});
    if (std.mem.indexOf(u8, text, "quick")) |index| {
        std.debug.print("'quick' 在索引 {d} 处找到\n", .{index});
    }
    if (std.mem.indexOf(u8, text, "slow")) |index| {
        std.debug.print("'slow' 在索引 {d} 处找到\n", .{index});
    } else {
        std.debug.print("'slow' 未找到\n", .{});
    }

    // 前缀和后缀
    std.debug.print("\n=== 前缀/后缀 ===\n", .{});
    std.debug.print(" 以 'The' 开头: {}\n", .{std.mem.startsWith(u8, text, "The")});
    std.debug.print(" 以 'jumps' 结尾: {}\n", .{std.mem.endsWith(u8, text, "jumps")});
}
```

输出：

```
=== 字符串操作 ===

原始: The quick brown fox jumps
第一个词: The
最后一个词: jumps

=== 比较 ===
'apple' == 'apple': true
'apple' == 'banana': false

=== 搜索 ===
'quick' 在索引 4 处找到
'slow' 未找到
```

```
=== 前缀/后缀 ===
以 'The' 开头: true
以 'jumps' 结尾: true
```

多行和转义序列

```zig
const std = @import("std");

pub fn main() void {
    // 使用 \\ 的多行字符串
    const poem =
        \\Roses are red,
        \\Violets are blue,
        \\Zig is awesome,
        \\And so are you!
    ;

    std.debug.print("=== 多行字符串 ===\n", .{});
    std.debug.print("{s}\n", .{poem});

    // 转义序列
    std.debug.print("\n=== 转义序列 ===\n", .{});
    const escaped = "Tab:\tNewline:\nBackslash:\\Quote:\"";
    std.debug.print("{s}\n", .{escaped});

    // 编译时连接
    std.debug.print("\n=== 连接 ===\n", .{});
    const first = "Hello, ";
    const second = "World!";
    const combined = first ++ second;  // 仅编译时
    std.debug.print("{s}\n", .{combined});
}
```

输出：

```
=== 多行字符串 ===
Roses are red,
Violets are blue,
Zig is awesome,
And so are you!

=== 转义序列 ===
Tab:    Newline:
Backslash:\Quote:"

=== 连接 ===
Hello, World!
```

要点

1. 字符串是 **`[]const u8`** - 只是字节切片，没有什么特别的
2. 使用 **`std.mem`** 进行操作 - `eql`、`indexOf`、`startsWith` 等
3. **`++`** 连接仅限编译时 - 运行时不能连接

4. 默认 **UTF-8** - Zig 字符串是 UTF-8 编码的
5. **{s}** 格式说明符 - 用于打印字符串

第九章：结构体

结构体让你将相关数据分组到自定义类型中。它们是组织代码最重要的特性之一。

结构体内存布局

```
┌──────────────────────────────────────────────────────────────────────┐
│                              结构体布局                              │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│   const Point = struct {                                             │
│       x: i32,                                                        │
│       y: i32,                                                        │
│   };                                                                 │
│                                                                      │
│   Point{ .x = 10, .y = 20 } 的内存布局：                             │
│                                                                      │
│   地址：    0x100         0x104                                      │
│                                                                      │
│   字段：    ┌──────────────┬──────────────┐                         │
│   值：      │      x       │      y       │                         │
│             │      10      │      20      │                         │
│             └──────────────┴──────────────┘                         │
│             |← 4 字节 →| |← 4 字节 →|                                │
│                                                                      │
│   总大小：8 字节                                                     │
│   字段按声明顺序排列                                                 │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 定义一个结构体类型
const Point = struct {
    x: i32,
    y: i32,
};

// 带默认值的结构体
const Rectangle = struct {
    x: i32 = 0,
    y: i32 = 0,
    width: u32 = 100,
    height: u32 = 100,
};

pub fn main() void {
    std.debug.print("=== 创建结构体 ===\n\n", .{});
```

```zig
    // 用所有字段创建
    const p1 = Point{ .x = 10, .y = 20 };
    std.debug.print("p1: ({d}, {d})\n", .{ p1.x, p1.y });

    // 用默认值创建
    const r1 = Rectangle{};  // 全部使用默认值
    std.debug.print("r1: pos=({d},{d}) size={d}x{d}\n", .{ r1.x, r1.y, r1.width, r1.height });

    // 覆盖部分默认值
    const r2 = Rectangle{ .x = 50, .y = 50, .width = 200 };
    std.debug.print("r2: pos=({d},{d}) size={d}x{d}\n", .{ r2.x, r2.y, r2.width, r2.height });

    // 可变结构体
    var p2 = Point{ .x = 0, .y = 0 };
    p2.x = 100;
    p2.y = 200;
    std.debug.print("p2 (修改后): ({d}, {d})\n", .{ p2.x, p2.y });

    // 大小信息
    std.debug.print("\nPoint 的大小: {d} 字节\n", .{@sizeOf(Point)});
    std.debug.print("Rectangle 的大小: {d} 字节\n", .{@sizeOf(Rectangle)});
}
```

输出：

```
=== 创建结构体 ===

p1: (10, 20)
r1: pos=(0,0) size=100x100
r2: pos=(50,50) size=200x100
p2 (修改后): (100, 200)

Point 的大小: 8 字节
Rectangle 的大小: 16 字节
```

结构体方法

```zig
const std = @import("std");

const Circle = struct {
    x: f64,
    y: f64,
    radius: f64,

    // 方法 - 接受 self 作为参数
    pub fn area(self: Circle) f64 {
        return std.math.pi * self.radius * self.radius;
    }

    // 方法 - 接受指针，可以修改
    pub fn scale(self: *Circle, factor: f64) void {
        self.radius *= factor;
    }
```

```zig
    // 方法 - 计算到另一个圆的距离
    pub fn distanceTo(self: Circle, other: Circle) f64 {
        const dx = self.x - other.x;
        const dy = self.y - other.y;
        return @sqrt(dx * dx + dy * dy);
    }

    // 关联函数 - 没有 self 参数
    pub fn unit() Circle {
        return Circle{ .x = 0, .y = 0, .radius = 1 };
    }
};

pub fn main() void {
    std.debug.print("=== 结构体方法 ===\n\n", .{});

    var c1 = Circle{ .x = 0, .y = 0, .radius = 5 };

    std.debug.print(" 圆: 圆心 =({d},{d}), 半径 ={d}\n", .{ c1.x, c1.y, c1.radius });
    std.debug.print(" 面积: {d:.2}\n", .{c1.area()});

    // 用方法修改
    c1.scale(2);
    std.debug.print("scale(2) 后: 半径 ={d}, 面积 ={d:.2}\n", .{ c1.radius, c1.area() });

    // 两个圆之间的距离
    const c2 = Circle{ .x = 3, .y = 4, .radius = 1 };
    std.debug.print("\n两圆之间的距离: {d}\n", .{c1.distanceTo(c2)});

    // 关联函数（类似静态方法）
    const unit = Circle.unit();
    std.debug.print(" 单位圆半径: {d}\n", .{unit.radius});
}
```

输出：

```
=== 结构体方法 ===

圆: 圆心=(0,0), 半径=5
面积: 78.54

scale(2) 后: 半径=10, 面积=314.16

两圆之间的距离: 5
单位圆半径: 1
```

要点

1. 命名字段 - 用 `.fieldname` 访问
2. 默认值 - 字段可以有默认值
3. 方法 - 带有 `self` 参数的函数
4. 关联函数 - 没有 `self` 的函数（类似静态）
5. 指针方法 - 使用 `*Self` 来修改结构体

第十章：枚举

枚举定义了一个具有固定命名值集合的类型。它们非常适合表示选择或状态。

枚举基础

```
┌──────────────────────────────────────────────────────────────┐
│                          枚举概念                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   const Direction = enum { north, south, east, west };       │
│                                                              │
│   不使用魔法数字：                                           │
│       const NORTH = 0;  // 0 是什么？                        │
│       const SOUTH = 1;  // 容易混淆                          │
│                                                              │
│   使用有意义的名称：                                         │
│       const dir = Direction.north;  // 清晰！                │
│                                                              │
│   好处：                                                     │
│   • 自文档化的代码                                           │
│   • 编译时检查                                               │
│   • switch 必须处理所有情况                                  │
│   • 可以有方法                                               │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

const Color = enum {
    red,
    green,
    blue,
    yellow,
    purple,
};

const Direction = enum {
    north,
    south,
    east,
    west,

    // 枚举可以有方法！
    pub fn opposite(self: Direction) Direction {
        return switch (self) {
            .north => .south,
            .south => .north,
            .east => .west,
```

```
            .west => .east,
        };
    }

    pub fn isVertical(self: Direction) bool {
        return self == .north or self == .south;
    }
};

pub fn main() void {
    std.debug.print("=== 枚举基础 ===\n\n", .{});

    // 创建枚举值
    const favorite: Color = .blue;
    var current: Color = .red;

    std.debug.print(" 最爱: {}\n", .{favorite});
    std.debug.print(" 当前: {}\n", .{current});

    // 改变值
    current = .green;
    std.debug.print(" 改为: {}\n", .{current});

    // 比较
    if (favorite == .blue) {
        std.debug.print(" 蓝色是最爱的！\n", .{});
    }

    // 在枚举上使用 switch（必须处理所有情况）
    std.debug.print("\n=== SWITCH ===\n", .{});
    const message = switch (current) {
        .red => " 停！",
        .green => " 走！",
        .blue => " 酷",
        .yellow => " 注意",
        .purple => " 皇家",
    };
    std.debug.print("{} 的消息: {s}\n", .{ current, message });

    // 方法
    std.debug.print("\n=== 方法 ===\n", .{});
    const dir: Direction = .north;
    std.debug.print(" 方向: {}\n", .{dir});
    std.debug.print(" 相反方向: {}\n", .{dir.opposite()});
    std.debug.print(" 是垂直的: {}\n", .{dir.isVertical()});
}
```

输出：

```
=== 枚举基础 ===

最爱: Color.blue
当前: Color.red
改为: Color.green
```

```
蓝色是最爱的！

=== SWITCH ===
Color.green 的消息: 走！

=== 方法 ===
方向: Direction.north
相反方向: Direction.south
是垂直的: true
```

要点

1. 穷尽式 **switch** - 必须处理每个枚举值
2. 枚举上的方法 - 枚举可以有自己的方法
3. 简短语法 - 使用 `.value` 而不是 `EnumType.value`
4. 类型安全 - 不能混用不同的枚举类型

第十一章：联合体

联合体可以持有多种类型中的一种，但一次只能持有一种。标记联合体知道哪个变体是活动的。

标记联合体概念

```
┌──────────────────────────────────────────────────────────────────┐
│                          标记联合体                              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   const Value = union(enum) {                                    │
│       int: i64,                                                  │
│       float: f64,                                                │
│       text: []const u8,                                          │
│   };                                                             │
│                                                                  │
│   一次只有一个变体是活动的：                                      │
│                                                                  │
│   ┌──────┬──────────────────────────────┐                       │
│   │ 标签 │            载荷              │                       │
│   ├──────┼──────────────────────────────┤                       │
│   │ int  │ 42 (i64)                     │   ← 活动              │
│   ├──────┼──────────────────────────────┤                       │
│   │float │ ──────────── (未使用)        │                       │
│   ├──────┼──────────────────────────────┤                       │
│   │text  │ ──────────── (未使用)        │                       │
│   └──────┴──────────────────────────────┘                       │
│                                                                  │
│   标签告诉我们当前存储的是哪个变体。                              │
│   大小 = max(载荷大小) + 标签大小                                 │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

const Value = union(enum) {
    int: i64,
    float: f64,
    text: []const u8,
    none: void,

    pub fn describe(self: Value) void {
        switch (self) {
            .int => |n| std.debug.print(" 整数: {d}\n", .{n}),
            .float => |f| std.debug.print(" 浮点数: {d:.2}\n", .{f}),
            .text => |s| std.debug.print(" 文本: {s}\n", .{s}),
            .none => std.debug.print(" 空\n", .{}),
        }
    }

    pub fn isNumeric(self: Value) bool {
        return switch (self) {
            .int, .float => true,
            .text, .none => false,
        };
    }
};

pub fn main() void {
    std.debug.print("=== 标记联合体 ===\n\n", .{});

    // 创建不同的变体
    const values = [_]Value{
        .{ .int = 42 },
        .{ .float = 3.14159 },
        .{ .text = "Hello" },
        .none,
    };

    for (values) |v| {
        std.debug.print(" 是数值: {}, ", .{v.isNumeric()});
        v.describe();
    }

    // 检查活动变体
    std.debug.print("\n=== 检查变体 ===\n", .{});
    const val: Value = .{ .int = 100 };

    if (val == .int) {
        std.debug.print(" 是整数！\n", .{});
    }

    // 带捕获的 switch
    switch (val) {
        .int => |n| std.debug.print(" 值是: {d}\n", .{n}),
        else => std.debug.print(" 不是整数\n", .{}),
```

```
    }
}
```

输出：

```
=== 标记联合体 ===

是数值: true, 整数: 42
是数值: true, 浮点数: 3.14
是数值: false, 文本: Hello
是数值: false, 空

=== 检查变体 ===
是整数！
值是: 100
```

要点

1. 一次一个变体 - 只有一个类型是活动的
2. 标签跟踪活动变体 - `union(enum)` 添加自动标签
3. 带捕获的 **switch** - 用 `|value|` 提取值
4. 内存高效 - 大小是最大载荷 + 标签，而不是所有的总和

第十二章：控制流

Zig 提供标准的控制流：`if`、`switch`、`while` 和 `for`。每个都可以作为表达式使用。

If 表达式

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== IF 表达式 ===\n\n", .{});

    const x: i32 = 42;

    // 基本的 if 语句
    if (x > 0) {
        std.debug.print("x 是正数\n", .{});
    }

    // If-else
    if (x % 2 == 0) {
        std.debug.print("x 是偶数\n", .{});
    } else {
        std.debug.print("x 是奇数\n", .{});
    }

    // If-else 链
    const score: u32 = 85;
    const grade = if (score >= 90)
```

```
        "A"
    else if (score >= 80)
        "B"
    else if (score >= 70)
        "C"
    else
        "F";

    std.debug.print(" 分数 {d} = 等级 {s}\n", .{ score, grade });

    // If 作为表达式（类似三元运算符）
    const abs_x = if (x >= 0) x else -x;
    std.debug.print("{d} 的绝对值: {d}\n", .{ x, abs_x });

    const sign = if (x > 0) " 正" else if (x < 0) " 负" else " 零";
    std.debug.print("{d} 是 {s}\n", .{ x, sign });
}
```

输出：

```
=== IF 表达式 ===

x 是正数
x 是偶数
分数 85 = 等级 B
42 的绝对值: 42
42 是 正
```

Switch 表达式

```
┌──────────────────────────────────────────────────────────────┐
│                        Switch 特性                           │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   • 穷尽：必须处理所有可能的值                                │
│   • 表达式：返回一个值                                        │
│   • 范围：用 ... 匹配一个范围                                 │
│   • 多个：用 , 匹配多个值                                     │
│   • 捕获：用 |x| 捕获值                                       │
│                                                              │
│   switch (value) {                                           │
│       1 => "一",            // 单个值                         │
│       2, 3 => "二或三",     // 多个值                         │
│       4...10 => "四到十",   // 范围                           │
│       else => "其他",       // 兜底                           │
│   }                                                          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```
const std = @import("std");

pub fn main() void {
    std.debug.print("=== SWITCH ===\n\n", .{});
```

```
    const day: u8 = 3;
    const name = switch (day) {
        1 => " 星期一",
        2 => " 星期二",
        3 => " 星期三",
        4 => " 星期四",
        5 => " 星期五",
        6, 7 => " 周末",
        else => " 无效",
    };
    std.debug.print(" 第 {d} 天是 {s}\n", .{ day, name });

    // switch 中的范围
    const age: u32 = 25;
    const category = switch (age) {
        0...12 => " 儿童",
        13...19 => " 青少年",
        20...64 => " 成人",
        65...120 => " 老年人",
        else => " 无效年龄",
    };
    std.debug.print(" 年龄 {d}: {s}\n", .{ age, category });
}
```

输出：

```
=== SWITCH ===

第 3 天是 星期三
年龄 25: 成人
```

循环

```
const std = @import("std");

pub fn main() void {
    // ========================================
    // WHILE 循环
    // ========================================
    std.debug.print("=== WHILE 循环 ===\n", .{});

    var i: u32 = 0;
    while (i < 5) {
        std.debug.print("{d} ", .{i});
        i += 1;
    }
    std.debug.print("\n", .{});

    // 带 continue 表达式的 while
    var j: u32 = 0;
    while (j < 10) : (j += 1) {
        if (j % 2 != 0) continue;
        std.debug.print("{d} ", .{j});
```

```zig
    }
    std.debug.print("\n", .{});

    // ========================================
    // FOR 循环
    // ========================================
    std.debug.print("\n=== FOR 循环 ===\n", .{});

    const numbers = [_]i32{ 10, 20, 30, 40, 50 };

    // 简单遍历
    for (numbers) |n| {
        std.debug.print("{d} ", .{n});
    }
    std.debug.print("\n", .{});

    // 带索引
    for (numbers, 0..) |n, idx| {
        std.debug.print("[{d}]={d} ", .{ idx, n });
    }
    std.debug.print("\n", .{});

    // 范围遍历
    std.debug.print(" 范围: ", .{});
    for (0..5) |i2| {
        std.debug.print("{d} ", .{i2});
    }
    std.debug.print("\n", .{});
}
```

输出：

```
=== WHILE 循环 ===
0 1 2 3 4
0 2 4 6 8

=== FOR 循环 ===
10 20 30 40 50
[0]=10 [1]=20 [2]=30 [3]=40 [4]=50
范围: 0 1 2 3 4
```

要点

1. 表达式，而非语句 - 控制流可以返回值
2. 穷尽式 **switch** - 必须处理所有情况
3. 没有 **++** 运算符 - 使用 += 1 或 continue 表达式
4. **for** 捕获值 - `|value|` 或 `|value, index|`
5. 标签块 - 使用 `blk: { break :blk value; }` 处理复杂表达式

第十三章：函数

函数是可重用的代码块。Zig 函数可以有编译时参数，使它们非常灵活。

函数解析

```
┌──────────────────────────────────────────────────────────────┐
│                          函数解析                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   fn functionName(param1: Type1, param2: Type2) ReturnType   │
│   │       │            │                        │            │
│   │       │            └── 参数 ─────────────────┘            │
│   │       │                                                  │
│   │       └── 函数名（camelCase）                            │
│   │                                                          │
│   └── fn 关键字                                              │
│                                                              │
│   pub fn add(a: i32, b: i32) i32 {                           │
│       return a + b;                                          │
│   }                                                          │
│   │                       │                                  │
│   └── pub = 可见          └── return 语句                    │
│       在模块外部                                             │
│                                                              │
│   特殊返回类型：                                             │
│   • void     - 没有返回值                                    │
│   • !T       - 可以返回错误或 T                              │
│   • ?T       - 可以返回 null 或 T                            │
│   • noreturn - 函数永不返回                                  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 简单函数
fn add(a: i32, b: i32) i32 {
    return a + b;
}

// void 返回 - 没有返回值
fn greet(name: []const u8) void {
    std.debug.print(" 你好, {s}!\n", .{name});
}

// 通过结构体返回多个值
fn divmod(num: i32, den: i32) struct { quotient: i32, remainder: i32 } {
    return .{
        .quotient = @divTrunc(num, den),
        .remainder = @mod(num, den),
    };
}

// 递归函数
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
```

```zig
    return n * factorial(n - 1);
}

pub fn main() void {
    std.debug.print("=== 函数 ===\n\n", .{});

    // 简单调用
    const sum = add(5, 3);
    std.debug.print("5 + 3 = {d}\n", .{sum});

    greet("Zig");

    // 多个返回值
    const result = divmod(17, 5);
    std.debug.print("17 / 5 = {d} 余数 {d}\n", .{ result.quotient, result.remainder });

    // 递归
    std.debug.print("5! = {d}\n", .{factorial(5)});
    std.debug.print("10! = {d}\n", .{factorial(10)});
}
```

输出：

```
=== 函数 ===

5 + 3 = 8
你好, Zig!
17 / 5 = 3 余数 2
5! = 120
10! = 3628800
```

要点

1. 函数使用 **camelCase** - Zig 约定
2. 显式类型 - 所有参数必须有类型
3. 单一返回 - 使用结构体返回多个值
4. 没有重载 - 每个名称一个函数（使用 comptime 实现泛型）

第十四章：错误处理

Zig 有内置的错误处理，使用错误联合体。错误是值，不是异常。

错误联合体概念

```
┌──────────────────────────────────────────────────────────────┐
│                          错误联合体                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    普通函数：fn divide(a: i32, b: i32) i32                   │
│    错误函数：fn divide(a: i32, b: i32) !i32                  │
│                                                    │         │
```

```
│                    可以返回错误或值 ──────────────────┘     │
│                                                            │
│   错误联合体：错误或值                                      │
│                                                            │
│   ┌──────────────────┬──────────────────┐                  │
│   │      错误        │      值          │                  │
│   │  (如果失败)      │  (如果成功)      │                  │
│   └──────────────────┴──────────────────┘                  │
│                                                            │
│   处理方式：                                                │
│   • try   – 将错误传播给调用者                              │
│   • catch – 本地处理错误                                    │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 定义错误集
const MathError = error{
    DivisionByZero,
    NegativeNumber,
};

// 可能失败的函数
fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

fn sqrt(x: i32) MathError!i32 {
    if (x < 0) return error.NegativeNumber;
    var r: i32 = 0;
    while (r * r <= x) : (r += 1) {}
    return r - 1;
}

pub fn main() void {
    std.debug.print("=== 错误处理 ===\n\n", .{});

    // 方法 1：用 if 检查
    if (divide(10, 2)) |result| {
        std.debug.print("10 / 2 = {d}\n", .{result});
    } else |err| {
        std.debug.print(" 错误: {}\n", .{err});
    }

    // 方法 2：catch 带默认值
    const safe = divide(10, 0) catch 0;
    std.debug.print("10 / 0 (带默认值): {d}\n", .{safe});

    // 方法 3：catch 带处理
    const result = divide(10, 0) catch |err| blk: {
        std.debug.print(" 捕获: {}\n", .{err});
```

```
        break :blk -1;
    };
    std.debug.print(" 结果: {d}\n", .{result});

    // 方法 4：catch unreachable（断言没有错误）
    const guaranteed = divide(20, 4) catch unreachable;
    std.debug.print("20 / 4 = {d}\n", .{guaranteed});
}
```

输出：

```
=== 错误处理 ===

10 / 2 = 5
10 / 0 (带默认值): 0
捕获: error.DivisionByZero
结果: -1
20 / 4 = 5
```

使用 try 进行错误传播

```
const std = @import("std");

const MathError = error{ DivisionByZero, NegativeNumber };

fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

fn sqrt(x: i32) MathError!i32 {
    if (x < 0) return error.NegativeNumber;
    var r: i32 = 0;
    while (r * r <= x) : (r += 1) {}
    return r - 1;
}

// try 自动传播错误
fn calculate(a: i32, b: i32) MathError!i32 {
    const quotient = try divide(a, b);  // 如果出错则返回
    const root = try sqrt(quotient);     // 如果出错则返回
    return root;
}

pub fn main() void {
    std.debug.print("=== TRY 传播 ===\n\n", .{});

    if (calculate(100, 4)) |result| {
        std.debug.print("calculate(100, 4) = {d}\n", .{result});
    } else |err| {
        std.debug.print(" 错误: {}\n", .{err});
    }
```

```zig
    if (calculate(100, 0)) |result| {
        std.debug.print("calculate(100, 0) = {d}\n", .{result});
    } else |err| {
        std.debug.print("calculate(100, 0) 错误: {}\n", .{err});
    }
}
```

输出：

```
=== TRY 传播 ===

calculate(100, 4) = 5
calculate(100, 0) 错误: error.DivisionByZero
```

要点

1. 错误是值 - 不是异常，是普通的返回值
2. **try** 传播 - 如果出错立即返回错误
3. **catch** 处理 - 提供默认值或本地处理
4. 自定义错误集 - 定义你自己的错误类型
5. **errdefer** - 仅在错误路径上清理

第十五章：可选类型

可选类型表示可能不存在的值。它们比空指针更安全。

可选类型概念

```
┌──────────────────────────────────────────────────────────────┐
│                          可选类型                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ?T 表示 "也许是 T" - 要么是 T 值要么是 null                  │
│                                                              │
│   const maybe: ?i32 = 42;    // 有值                         │
│   const empty: ?i32 = null;  // 无值                         │
│                                                              │
│    ┌──────────────────────────────────────┐                  │
│    │  ?i32 有值                           │                  │
│    │   ┌──────────┬──────────────┐        │                  │
│    │   │ has_val  │    value     │        │                  │
│    │   │  true    │     42       │        │                  │
│    │   └──────────┴──────────────┘        │                  │
│    └──────────────────────────────────────┘                  │
│                                                              │
│    ┌──────────────────────────────────────┐                  │
│    │  ?i32 无值 (null)                    │                  │
│    │   ┌──────────┬──────────────┐        │                  │
│    │   │ has_val  │    value     │        │                  │
│    │   │  false   │ (未定义)     │        │                  │
```

```
│    │  └──────────┴──────────────┘                │
│    └──────────────────────────────────────────┘
│
│    解包方式：
│    • if (opt) |val| { }    - 安全解包
│    • opt orelse default    - 提供默认值
│    • opt.?                 - 断言非 null（如果是则崩溃）
│
└──────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// 返回可选类型的函数
fn find(haystack: []const u8, needle: u8) ?usize {
    for (haystack, 0..) |byte, index| {
        if (byte == needle) return index;
    }
    return null;
}

fn getElement(arr: []const i32, index: usize) ?i32 {
    if (index >= arr.len) return null;
    return arr[index];
}

pub fn main() void {
    std.debug.print("=== 可选类型 ===\n\n", .{});

    // 创建可选类型
    var maybe: ?i32 = 42;
    std.debug.print("maybe = {?d}\n", .{maybe});

    maybe = null;
    std.debug.print("maybe (null) = {?d}\n", .{maybe});

    // If 解包模式
    const text = "Hello, World!";
    if (find(text, 'W')) |index| {
        std.debug.print("'W' 在索引 {d} 处找到\n", .{index});
    }

    if (find(text, 'Z')) |index| {
        std.debug.print("'Z' 在索引 {d} 处找到\n", .{index});
    } else {
        std.debug.print("'Z' 未找到\n", .{});
    }

    // orelse - 提供默认值
    const val1: ?i32 = 42;
    const val2: ?i32 = null;

    std.debug.print("\nval1 orelse 0: {d}\n", .{val1 orelse 0});
    std.debug.print("val2 orelse -1: {d}\n", .{val2 orelse -1});
```

```
    // 安全的数组访问
    const numbers = [_]i32{ 10, 20, 30 };
    std.debug.print("\nnumbers[1] = {?d}\n", .{getElement(&numbers, 1)});
    std.debug.print("numbers[10] = {?d}\n", .{getElement(&numbers, 10)});
}
```

输出：

```
=== 可选类型 ===

maybe = 42
maybe (null) = null
'W' 在索引 7 处找到
'Z' 未找到

val1 orelse 0: 42
val2 orelse -1: -1

numbers[1] = 20
numbers[10] = null
```

要点

1. **?T** = 也许是 **T** - 值或 null
2. **if (opt) |val|** - 带载荷捕获的安全解包
3. **orelse** - 如果为 null 则提供默认值
4. **.?** 运算符 - 解包（如果为 null 则恐慌）
5. 没有空指针异常 - 必须显式处理 null

第十六章：编译时

Comptime 在编译时运行代码。这使得无需宏即可实现强大的元编程。

```
const std = @import("std");

// 编译时常量
const KILOBYTE = 1024;
const MEGABYTE = KILOBYTE * 1024;
const GIGABYTE = MEGABYTE * 1024;

// 编译时函数
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// 在编译时预计算
const FACT_10 = factorial(10);

pub fn main() void {
    std.debug.print("=== 编译时 ===\n\n", .{});
```

```
    std.debug.print("1 KB = {d} 字节\n", .{KILOBYTE});
    std.debug.print("1 MB = {d} 字节\n", .{MEGABYTE});
    std.debug.print("1 GB = {d} 字节\n", .{GIGABYTE});

    std.debug.print("\n10! = {d} (在编译时计算！)\n", .{FACT_10});

    // comptime 块
    const sum = comptime blk: {
        var total: u32 = 0;
        for (1..11) |i| {
            total += @as(u32, @intCast(i));
        }
        break :blk total;
    };
    std.debug.print("1-10 的和 = {d} (编译时)\n", .{sum});

    // 类型反射
    std.debug.print("\n类型信息:\n", .{});
    std.debug.print("  i32 大小: {d} 字节\n", .{@sizeOf(i32)});
    std.debug.print("  i64 大小: {d} 字节\n", .{@sizeOf(i64)});
}
```

输出：

```
=== 编译时 ===

1 KB = 1024 字节
1 MB = 1048576 字节
1 GB = 1073741824 字节

10! = 3628800 (在编译时计算！)
1-10 的和 = 55 (编译时)

类型信息:
  i32 大小: 4 字节
  i64 大小: 8 字节
```

第十七章：泛型

泛型让你编写适用于任何类型的代码。

```
const std = @import("std");

// 泛型函数
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn swap(comptime T: type, a: *T, b: *T) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
```

```
}

// 泛型结构体
fn Stack(comptime T: type) type {
    return struct {
        const Self = @This();
        items: [100]T = undefined,
        count: usize = 0,

        pub fn push(self: *Self, item: T) void {
            self.items[self.count] = item;
            self.count += 1;
        }

        pub fn pop(self: *Self) ?T {
            if (self.count == 0) return null;
            self.count -= 1;
            return self.items[self.count];
        }
    };
}

pub fn main() void {
    std.debug.print("=== 泛型 ===\n\n", .{});

    // 泛型函数
    std.debug.print("max(i32, 5, 10) = {d}\n", .{max(i32, 5, 10)});
    std.debug.print("max(f64, 3.14, 2.71) = {d}\n", .{max(f64, 3.14, 2.71)});

    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\n交换前: x={d}, y={d}\n", .{ x, y });
    swap(i32, &x, &y);
    std.debug.print(" 交换后: x={d}, y={d}\n", .{ x, y });

    // 泛型结构体
    std.debug.print("\n=== 泛型栈 ===\n", .{});
    var stack = Stack(i32){};
    stack.push(10);
    stack.push(20);
    stack.push(30);

    while (stack.pop()) |val| {
        std.debug.print(" 弹出: {d}\n", .{val});
    }
}
```

输出：

```
=== 泛型 ===

max(i32, 5, 10) = 10
max(f64, 3.14, 2.71) = 3.14
```

```
交换前: x=100, y=200
交换后: x=200, y=100

=== 泛型栈 ===
弹出: 30
弹出: 20
弹出: 10
```

第十八章：内存管理

Zig 给你对内存分配的显式控制。

```zig
const std = @import("std");

pub fn main() !void {
    std.debug.print("=== 内存管理 ===\n\n", .{});

    // 栈内存（自动）
    var stack_var: i32 = 42;
    var stack_arr: [10]i32 = undefined;
    for (&stack_arr, 0..) |*item, i| {
        item.* = @as(i32, @intCast(i * 2));
    }
    std.debug.print(" 栈变量: {d}\n", .{stack_var});
    std.debug.print(" 栈数组: {any}\n", .{stack_arr});

    // 堆内存（手动）
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // 分配单个值
    const ptr = try allocator.create(i32);
    defer allocator.destroy(ptr);
    ptr.* = 100;
    std.debug.print("\n堆值: {d}\n", .{ptr.*});

    // 分配切片
    const slice = try allocator.alloc(i32, 5);
    defer allocator.free(slice);
    for (slice, 0..) |*item, i| {
        item.* = @as(i32, @intCast((i + 1) * 10));
    }
    std.debug.print(" 堆切片: {any}\n", .{slice});

    // ArrayList
    var list = std.ArrayList(i32).init(allocator);
    defer list.deinit();
    try list.append(1);
    try list.append(2);
    try list.append(3);
    std.debug.print("ArrayList: {any}\n", .{list.items});
```

```
}
```

输出：

```
=== 内存管理 ===

栈变量: 42
栈数组: { 0, 2, 4, 6, 8, 10, 12, 14, 16, 18 }

堆值: 100
堆切片: { 10, 20, 30, 40, 50 }
ArrayList: { 1, 2, 3 }
```

第十九章：测试

Zig 有内置的测试支持。

```zig
const std = @import("std");
const expect = std.testing.expect;
const expectEqual = std.testing.expectEqual;

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn factorial(n: u32) u32 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

test " 基本加法" {
    try expectEqual(@as(i32, 5), add(2, 3));
    try expectEqual(@as(i32, 0), add(-5, 5));
}

test " 阶乘" {
    try expectEqual(@as(u32, 1), factorial(0));
    try expectEqual(@as(u32, 1), factorial(1));
    try expectEqual(@as(u32, 120), factorial(5));
}

test "expect 示例" {
    try expect(add(1, 1) == 2);
    try expect(10 > 5);
}

pub fn main() void {
    std.debug.print(" 运行测试: zig test filename.zig\n", .{});
}
```

运行方式：`zig test filename.zig`

第二十章：C 语言互操作

Zig 可以直接调用 C 代码。

```zig
const std = @import("std");
const c = @cImport({
    @cInclude("math.h");
});

pub fn main() void {
    std.debug.print("=== C 互操作 ===\n\n", .{});

    // 调用 C 数学函数
    const x: c.double = 2.0;

    std.debug.print("sqrt({d}) = {d}\n", .{ x, c.sqrt(x) });
    std.debug.print("pow({d}, 3) = {d}\n", .{ x, c.pow(x, 3.0) });
    std.debug.print("sin({d}) = {d}\n", .{ x, c.sin(x) });
    std.debug.print("cos({d}) = {d}\n", .{ x, c.cos(x) });
}

// 导出函数供 C 调用
export fn zig_add(a: c_int, b: c_int) c_int {
    return a + b;
}
```

输出：

```
=== C 互操作 ===

sqrt(2) = 1.4142135623730951
pow(2, 3) = 8
sin(2) = 0.9092974268256817
cos(2) = -0.4161468365471424
```

附录 A：Zig 速查表

```
┌──────────────────────────────────────────────────────────────┐
│                          ZIG 速查表                          │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  变量                                                        │
│  ────────────────────────────────────────────────────        │
│  const x: i32 = 42;          // 不可变                       │
│  var y: i32 = 0;             // 可变                         │
│  var z: i32 = undefined;     // 未初始化                     │
│                                                              │
│  类型                                                        │
│  ────────────────────────────────────────────────────        │
│  i8, i16, i32, i64, i128     // 有符号整数                   │
│  u8, u16, u32, u64, u128     // 无符号整数                   │
```

```
f16, f32, f64, f128       // 浮点数
bool                      // 布尔
[]T                       // 切片
[N]T                      // 数组
*T                        // 指针
?T                        // 可选
!T 或 E!T                 // 错误联合

控制流
──────────────────────────────────────────────────────
if (cond) { } else { }
switch (x) { val => {}, else => {} }
while (cond) : (cont) { }
for (slice) |item| { }
for (slice, 0..) |item, i| { }

函数
──────────────────────────────────────────────────────
fn name(param: T) ReturnType { }
fn name(param: T) !ReturnType { }  // 可能出错

错误处理
──────────────────────────────────────────────────────
try expression           // 传播错误
catch |err| { }          // 处理错误
orelse default           // 解包可选
if (opt) |val| { }       // 解包可选

内存
──────────────────────────────────────────────────────
allocator.create(T)      // 分配一个
allocator.alloc(T, n)    // 分配多个
allocator.destroy(ptr)   // 释放一个
allocator.free(slice)    // 释放多个
defer cleanup();         // 作用域退出时运行

常用内置函数
──────────────────────────────────────────────────────
@import("name")          // 导入模块
@sizeOf(T)               // 类型大小
@typeInfo(T)             // 类型信息
@intCast(val)            // 整数转换
@floatCast(val)          // 浮点转换
```

附录 B：常用模式

常用模式

可选类型解包

```
if (optional) |value| {
    // 使用 value
} else {
    // 处理 null
}

const val = optional orelse default;
const val = optional orelse return error.NotFound;
```

错误处理

```
const result = try riskyOperation();

const result = riskyOperation() catch |err| {
    log.err("失败: {}", .{err});
    return err;
};

const result = riskyOperation() catch default;
```

资源清理

```
const file = try fs.openFile(path);
defer file.close();
// 使用 file...
// close() 自动调用！

const ptr = try allocator.create(T);
errdefer allocator.destroy(ptr);  // 仅在出错时
try initializePtr(ptr);
```

关于作者

JW Lee 是一位热衷于系统编程的软件开发者，致力于让复杂的主题对初学者变得易于理解。本书代表了他为 Zig 编程语言创建最全面且最易懂的入门教程的努力。

简易 *Zig - Zig* 编程语言零基础入门指南

版权所有 *2024 JW Lee*

本书中所有示例均在 *MIT* 许可证下提供。